EE5003 - MSc Project Report

National University of Singapore

Department of Electrical & Computer Engineering

ON POLICY MULTI-AGENT REINFORCEMENT LEARNING FOR 6G EDGE-CLOUD COMPUTING

Author:
Kan Ziyang (ID: A0326643A)
e1538474@u.nus.edu

Supervisor:
Prof. Tham Chen Khong

Examiner:
Prof. Soh Wee Seng

March 2026

Abstract

Task offloading in 6G edge-cloud systems requires joint decisions over heterogeneous computing resources and wireless bandwidth, while the environment evolves continuously because of mobility, load fluctuations, and link contention. Under a three-tier LOCAL–EDGE–CLOUD architecture, the coupled offloading and Physical Resource Block (PRB) allocation problem becomes a high-dimensional combinatorial optimization task for which static heuristics are difficult to tune across changing conditions. This report presents an on-policy multi-agent reinforcement learning framework based on Multi-Agent Proximal Policy Optimization (MAPPO) to address this challenge.

The proposed formulation models each task-generating edge device as an autonomous agent in a decentralized partially observable Markov decision process. The framework follows centralized training with decentralized execution: each device actor selects an offloading destination and a PRB allocation level from a dual-head discrete action space using only its local observation, while a centralized critic evaluates the global system state during training. To support this learning loop, PureEdgeSim is extended with PRB-aware wireless resource management, a distance-attenuated bandwidth model, a device-centric reinforcement learning interface, and a Java-Python co-simulation layer based on a TCP JSON protocol.

The implementation combines a Java discrete-event simulator and Python learning modules for both MAPPO and a shared-policy PPO baseline. Under the standard 160-device, 30-minute configuration, both learned policies substantially outperform heuristic schedulers. MAPPO reaches a three-seed mean task success rate of 99.9237%, PPO reaches 99.9436%, and both remain far above the strongest preserved heuristic baseline, ROUND_ROBIN, at 90.3869%. The finalized ablation results further clarify the design: removing agent embeddings causes only a small reliability drop to 99.8148%, whereas fixing PRB requests degrades mean success to 92.3822% and raises mean energy consumption to 309.2645 W. These results show that on-policy reinforcement learning is a practical direction for adaptive offloading control, that adaptive PRB allocation is essential to system quality, and that explicit multi-agent conditioning provides a smaller but still measurable benefit on top of joint offloading-and-radio optimization.

Keywords: Multi-Agent Reinforcement Learning, MAPPO, PPO, Task Offloading, PRB Allocation, 6G Edge-Cloud Computing, PureEdgeSim.

Acknowledgement

The author would like to thank the project supervisor, examiner, and colleagues in the Department of Electrical and Computer Engineering for their guidance, feedback, and support throughout this MSc project. Their discussions on mobile edge computing, reinforcement learning, and simulation methodology helped shape both the system design and the evaluation strategy reported in this work.

The author also acknowledges the open-source communities behind PureEdgeSim, CloudSim Plus, PyTorch, and the related research codebases that made rapid prototyping and reproducible experimentation possible. Their contributions provided a strong foundation for the development of the customized 6G edge-cloud simulation environment used in this project.

Declaration

I hereby declare that this report is my original work and that it has been written by me in its entirety. I have duly acknowledged all sources of information that have been used in the report.

Signature: _____

Name: _____

Date: _____

Contents

List of Figures

List of Tables

List of Abbreviations

Acronym	**Meaning**
6G	Sixth-generation wireless communication systems
CTDE	Centralized Training with Decentralized Execution
Dec-POMDP	Decentralized Partially Observable Markov Decision Process
DRL	Deep Reinforcement Learning
GAE	Generalized Advantage Estimation
JSON	JavaScript Object Notation
LAN	Local Area Network
MAPPO	Multi-Agent Proximal Policy Optimization
MARL	Multi-Agent Reinforcement Learning
MEC	Mobile Edge Computing
MIPS	Million Instructions Per Second
MLP	Multi-Layer Perceptron
PPO	Proximal Policy Optimization
PRB	Physical Resource Block
RL	Reinforcement Learning
TCP	Transmission Control Protocol
VM	Virtual Machine
WLAN	Wireless Local Area Network

1 Introduction

1.1 Overview

The evolution toward 6G is expected to support ultra-low latency services, massive device connectivity, and deeply integrated intelligent control across communication and computing infrastructure. In this setting, computation can no longer be treated as a cloud-only service. Instead, latency-sensitive and computation-intensive workloads must be distributed over a hierarchical edge-cloud continuum in which local devices, nearby edge data centers, and remote cloud servers cooperate under dynamic wireless and computing conditions [1–3].

This report focuses on a three-tier LOCAL–EDGE–CLOUD architecture in which task offloading decisions are tightly coupled with wireless **Physical Resource Block (PRB)** allocation. Although 6G standardization is still evolving, scarce schedulable radio resources will remain a core systems constraint in future mobile networks. In this report, a PRB-based abstraction, inherited from OFDM-based cellular systems such as 5G NR, is used to model uplink radio-resource contention in a 6G-oriented edge-cloud scenario. Under this abstraction, the practical difficulty is not only deciding where a task should be executed, but also deciding how much wireless resource should be committed to that decision. Sending a task to a powerful remote destination may reduce computation delay while increasing transmission delay and PRB contention; keeping too many tasks local preserves bandwidth but can overload resource-constrained devices. As the number of devices increases, these interactions create a high-dimensional, non-stationary, and strongly coupled control problem that is difficult to solve with static heuristics alone.

Motivated by this challenge, the project studies **on-policy** reinforcement learning for dynamic resource allocation in 6G edge-cloud computing. MAPPO is the main method studied in this report, while PPO serves as the closest controlled learning baseline under the same environment and dual-action design. The goal is not to claim in advance that multi-agent modeling is always superior, but to evaluate whether cooperative on-policy learning can improve Quality of Service (QoS) and overall system efficiency in a realistic simulation loop. The main contributions are summarized as follows.

(a) The joint task-offloading and PRB-allocation problem is formulated as a decentralized partially observable Markov decision process (Dec-POMDP) in which all task-generating devices act as decentralized agents.

(b) A MAPPO framework is designed under centralized training with decentralized execution (CTDE), combining per-device actors, agent-aware representations, and a centralized critic over the global simulator state.

(c) A shared-policy PPO baseline is implemented under the same environment interface, providing a controlled comparison against the proposed multi-agent design.

(d) PureEdgeSim is extended into a complete training, inference, and evaluation pipeline with PRB-aware networking, a distance-sensitive wireless model, telemetry logging, and Java–Python co-simulation over a TCP JSON protocol.

(e) The resulting framework is evaluated against heuristic and learning-based baselines using QoS and system-cost metrics, including task success rate, completion latency, energy consumption, and communication-resource usage.

1.2 Background

1.2.1 6G and Mobile Edge Computing

6G visions place intelligence, communication, and distributed computing into the same end-to-end system. Emerging applications such as immersive media, intelligent transportation, industrial automation, and large-scale IoT require both high-bandwidth connectivity and fast computation close to end users [1]. Mobile edge computing addresses this demand by pushing computation closer to users, thereby reducing backhaul pressure and end-to-end latency compared with cloud-only execution [2, 3]. In practice, however, edge resources are still limited and heterogeneous. A realistic deployment therefore requires collaboration among local devices, edge data centers, and the cloud rather than reliance on any single tier.

The connection between 6G and MEC is especially strong when radio-resource allocation is considered. Even if nearby edge servers are available, a task cannot benefit from offloading unless the wireless link is granted sufficient transmission resources. In practical systems, schedulable uplink radio resources directly determine the effective bandwidth available to each task and therefore influence transmission time, queueing pressure, deadline satisfaction, and the usefulness of remote execution itself. In this report, these wireless resources are represented through a PRB-based abstraction inspired by current OFDM-based cellular systems. For this reason, radio-resource scheduling is not a secondary networking detail but a core part of intelligent 6G-oriented edge-cloud orchestration.

This observation also changes the meaning of task offloading. In earlier formulations, offloading is often treated as a destination-selection problem or, at most, a decision over what fraction of a task should be sent to the edge. In a radio-resource-constrained 6G-oriented environment, however, a new generation of task-offloading methods must jointly consider **where** the task is executed and **how much wireless resource** is allocated to make that execution feasible and efficient. Reasonable PRB allocation is therefore essential to avoid turning a good compute decision into a poor end-to-end service outcome.

1.2.2 Traditional Task Offloading Methods

Classical task-offloading strategies often rely on heuristics, static priority rules, convex optimization, or other model-based formulations. These methods can work well when the environment is simple, stationary, and fully observable. Yet recent surveys and edge-offloading studies show that their assumptions become restrictive in dynamic multi-user systems, where task arrivals, wireless conditions, queue lengths, mobility, and resource availability change over time [4, 5]. In particular, many traditional formulations either treat communication delay as a fixed cost or abstract wireless capacity into a coarse scalar term. Such simplifications become increasingly problematic in 6G-oriented edge-cloud systems, where finite radio resources must be shared by many devices and where contention directly changes the practical value of an offloading decision.

This limitation is especially important for new-generation task offloading. A destination that appears optimal under static compute assumptions may become suboptimal when the uplink receives insufficient radio resources, because transmission time, congestion, and deadline misses can dominate the end-to-end outcome. Conversely, an apparently conservative local or nearby-edge decision may be preferable when wireless resources are scarce. As a result, task offloading in 6G-oriented systems should no longer be viewed as an isolated destination-selection problem; it must be treated as a joint communication-and-computation allocation problem in which reasonable PRB assignment is part of the scheduling logic rather than an afterthought.

1.2.3 Why Multi-Agent Reinforcement Learning

Reinforcement learning is attractive because it can optimize long-run behavior directly through repeated interaction with the environment. For edge-cloud orchestration, however, a single-agent formulation quickly becomes unwieldy as the number of devices and feasible actions grows. Multiple devices compete through shared wireless resources, shared edge queues, and limited cloud or edge compute resources, which makes the problem inherently interactive. One user obtaining more PRB-equivalent radio resources or occupying a preferred edge destination can immediately change the feasible and desirable actions for other users. This coupling makes the scheduling problem naturally multi-agent rather than merely large-scale.

Recent MEC literature therefore increasingly turns to cooperative MARL, hierarchical MARL, or other distributed learning schemes to coordinate multiple users under shared constraints [5–8]. MARL is particularly suitable here because each device can act on local information at decision time while training still exploits system-level information to learn cooperative behavior. In this report, QoS is the primary service-level objective and is reflected through task success rate, completion latency, and failure-related behavior. These metrics must be balanced against system cost, including energy usage, communication-resource occupation, and the training or inference overhead of the decision framework itself. This is precisely the type of coupled, dynamic control

problem for which on-policy MARL provides a strong and practically meaningful modeling choice.

1.3 Related Work

Recent literature on intelligent edge orchestration can be grouped into four closely related strands. The first strand studies reinforcement learning for task offloading and joint resource allocation in MEC. Lu **et al.** combine MADDPG and SAC to jointly optimize target-server selection and offloaded data amount in a mobile MEC system with wireless power transfer, using a two-agent decomposition per device (one for discrete server selection via Gumbel-Softmax, one for continuous offloading ratio) and a centralized critic that observes all agents' states and actions [6]. Ke **et al.** propose DeMADRL, a decentralized multi-agent DDQN scheme for partial task offloading and bandwidth allocation under time-varying channel conditions, discretizing the offloading ratio into 11 levels and the bandwidth ratio into 15 levels [7]. However, their agents are fully independent with no centralized coordination, and the evaluation scale is limited to 15 wireless nodes and 3 MEC servers. Sun and He further decompose the problem into high-level server selection and low-level offloading-ratio control using a hierarchical MAPPO framework with separate high-level and low-level actors [8]. Their work is the closest algorithmic reference to the present report, but it operates at a smaller scale (30 user equipments, 3 MEC servers), does not include a cloud tier, and does not model wireless resource allocation at the PRB level. Aghapour **et al.** decompose the offloading and resource allocation problem into two subproblems—offloading decisions by deep RL and compute-resource allocation by an improved Salp Swarm Algorithm—demonstrating that the two-part decomposition pattern is effective for CNN-layer distribution across IoT cloudlets [9]. A recent survey by Luo and Dai confirms that latency and energy remain the dominant optimization targets and that DDPG and DQN families are still the most frequently used algorithms, while PPO/MAPPO-based designs remain less common [4]. This gap in algorithm coverage further motivates the present study.

The second strand focuses on hierarchical or guided multi-agent decision making. Robles-Enciso and Skarmeta study task assignment across edge, fog, and cloud layers through a multi-layer reinforcement learning system in which edge agents can query an upper-layer fog agent with broader system knowledge [5]. Their work is especially relevant because it highlights the limited local observability of edge devices and the value of cooperative decision support in computing-continuum settings. In their 2023 journal version, the multi-layer guided RL achieves approximately 85% task success rate at 200 devices, compared with roughly 55% for the greedy baseline [5]. At the same time, their approach is based on tabular Q-learning with only 1728 discrete states and does not model explicit PRB-aware wireless contention. Cao **et al.** study a related but distinct problem—edge-to-edge service migration for load balancing—using a DR-DQN approach that jointly considers offloading demand and server reception capacity [10]. Their emphasis on inter-server load balancing complements the device-to-tier offloading

focus of the present report.

The third strand concerns simulation and evaluation infrastructure. PureEdgeSim provides a discrete-event simulation framework for cloud, edge, and mist computing environments and remains a practical basis for studying heterogeneous devices, mobility, and task orchestration [11]. More recently, Mechalikh **et al.** compared 24 edge simulators and found that PureEdgeSim achieves zero bugs, 91% code coverage, and 0% code duplication—the strongest code-quality profile among all evaluated tools [12]. EISim extends PureEdgeSim toward AI-driven management experiments [13], while the ML-RL-PureEdgeSim codebase provides an earlier RL-oriented integration around the simulator runtime [14]. The present project builds on that direction and pushes the simulator toward explicit PRB-aware networking, device-agent interaction, and a bidirectional Java–Python learning pipeline.

The fourth strand concerns the choice of on-policy MARL itself and its connection to radio-resource optimization. In cooperative multi-agent settings, Yu **et al.** show that PPO-based methods can be unexpectedly strong in both final return and sample efficiency, especially when combined with centralized critics, value normalization, and carefully controlled policy updates [15]. Their systematic study identifies five critical implementation factors—value normalization, value-function input representation, training data usage, clipping range, and batch size—that directly inform the hyperparameter choices in this report. On the radio-resource side, Giwa **et al.** demonstrate that PPO outperforms heuristic baselines for joint power, bandwidth, and scheduling allocation in heterogeneous wireless networks, reaching higher overall reward than TD3 despite slower initial convergence [16]. Together, these results support the use of MAPPO and PPO as serious methods for cooperative edge-cloud control with explicit wireless resource management.

Taken together, prior work strongly supports learning-based offloading and increasingly recognizes the value of multi-agent or hierarchical control. However, several gaps remain. First, most published formulations still optimize destination selection or offloading ratio under simplified communication assumptions—fixed bandwidth, no PRB modeling, no distance-dependent attenuation. Second, the closest MAPPO-based work [8] operates at a much smaller scale and does not include a cloud tier or wireless resource allocation. Third, the PureEdgeSim-based RL studies [5, 17] use tabular Q-learning and do not model radio-resource contention. By contrast, this report models explicit PRB-aware wireless contention inside a customized PureEdgeSim environment and studies the coupled decision over execution destination and PRB reservation under a unified on-policy learning pipeline at a scale of 160 devices generating over 67,000 decisions per episode.

Tab. 1 summarizes the key differences between this work and the most closely related studies.

Study	Algorithm	Tiers	Agents	PRB/BW action	Critic	Simulator	Scale
Lu et al. [6]	MADDPG+SAC	Local+Edge	per-device	No	Centralized (joint)	Custom	variable
Ke et al. [7]	DDQN	Local+Edge+Cloud	per-device	BW ratio (15 bins)	None (independent)	Custom	15 WNs
Sun & He [8]	Hierarchical MAPPO	Local+Edge	per-UE	No	Centralized (global)	Custom	30 UEs
Robles [5]	Tabular Q-learning	Edge+Fog+Cloud	per-device	No	Multi-layer query	PureEdgeSim	10–200
Aghapour [9]	DRL + SSA	Local+Edge+Cloud	centralized	Compute (SSA)	N/A	Custom	multi-user
Giwa [16]	PPO / TD3	HetNet (no MEC)	centralized	Power+BW+sched.	N/A	Custom	multi-BS
This work	**MAPPO / PPO**	**Local+Edge+Cloud**	**per-device**	**PRB (8 bins)**	**Centralized (global+telemetry)**	**PureEdgeSim**	**160 devices**

Table 1: Comparison of this work with the most closely related studies. PRB/BW action indicates whether the method includes an explicit wireless resource allocation decision.

1.4 Report Organization

The remainder of this report is organized as follows. Section 2 describes the simulation environment, including the open-source PureEdgeSim foundation, the project-specific extensions, and the instantiated experimental configuration. Section 3 formalizes the system model and problem formulation, covering the network architecture, task model, communication model, computation model, energy model, and the Dec-POMDP formulation. Section 4 presents the MAPPO framework, observation and action design, network architecture, reward function, and training algorithm. Section 5 reports the experimental evaluation, including the standard MAPPO training and evaluation, device-count scalability, ablation studies, a multi-algorithm offline comparison, and the MAPPO–PPO analysis. Section 6 summarizes the findings, discusses limitations, and outlines directions for future work.

2 Simulation Environment

2.1 Open-Source Foundation

The simulator used in this report is not a system built from scratch. It is a research extension of the open-source PureEdgeSim line, which was introduced by Mechalikh **et al.** as a modular discrete-event framework for cloud, edge, and mist computing evaluation [11, 18]. The original motivation of PureEdgeSim is closely aligned with the needs of this project: heterogeneous devices, mobility, latency-sensitive applications, and energy-aware orchestration are difficult to study analytically, but can be explored systematically in a simulator that already models tasks, data centers, networking, and event scheduling.

The 2021 PureEdgeSim paper is particularly important because it describes the simulator as a layered and modular environment built on top of CloudSim Plus, with dedicated managers for simulation control, data-center generation, task generation, mobility, networking, and orchestration [11, 19]. That architecture is precisely what makes PureEdgeSim a good modification target: it already provides the event-driven backbone, CSV logging, infrastructure configuration, and heterogeneous device support required for edge-cloud studies, so the project can focus on extending the scheduler and network logic rather than reimplementing an entire simulator stack.

The later comparative study by Mechalikh **et al.** evaluates 24 edge simulators and identifies PureEdgeSim as one of the few tools that remained regularly maintained while also showing strong scalability and reliability characteristics [12]. This is an important justification for the simulator choice in a report like this one. Since the project goal is to study a new scheduling method rather than defend a new simulator, using a maintained and open-source research platform is methodologically stronger than inventing a custom environment with no external baseline. The codebase used in this report follows the CloudSim-Plus-based PureEdgeSim line, with PureEdgeSim 4.2.0 and CloudSim Plus 6.2.7 as the immediate software foundation.

2.2 Relation to Prior PureEdgeSim-Based RL Systems

The two papers by Robles-Enciso and Skarmeta are especially relevant because they show how PureEdgeSim can be extended beyond heuristic orchestration and used as a reinforcement-learning testbed [5, 17]. In the 2022 conference paper and the 2023 journal version, the authors modify PureEdgeSim v4.2 to evaluate greedy, single-layer RL, and multi-layer guided RL solutions for task offloading in a computing continuum. Their evaluation uses a $200\,\mathrm{m} \times 200\,\mathrm{m}$ scenario, an edge–fog–cloud hierarchy, repeated random simulations, and metrics such as task success rate, average total time, latency failures, CPU usage, and energy consumption. These papers are therefore strong evidence that PureEdgeSim is not only suitable for classical heuristic studies, but also suitable for repeated RL experiments over heterogeneous multi-tier infrastructures.

At the same time, the environment required by this report is substantially different from the one used in those papers. Robles-Enciso and Skarmeta study task assignment among local, edge, fog, and cloud processing nodes, and their learning agents operate on offloading choices without an explicit radio-resource action space [5, 17]. By contrast, the present work targets a LOCAL–EDGE–CLOUD architecture and treats wireless Physical Resource Block (PRB) reservation as part of the decision itself. The project therefore reuses the open-source event-driven simulator foundation validated by those studies, but it extends the environment toward joint communication-and-computation control and deep on-policy learning.

The main project-specific extensions are summarized in Fig. 1. Compared with the published PureEdgeSim architecture and the earlier PureEdgeSim-based RL studies, this report introduces the following additional components.

(a) **PRB-aware network manager.** The default network layer is extended with a global PRB pool, fixed reservations for RL-selected actions, and a dynamic allocator for non-RL baselines.

(b) **Device-level decision support.** `DeviceAgentDecisionSupport` constructs local observations, destination features, feasibility masks, PRB mappings, fallback handling, and rewards.

(c) **Unified RL orchestration path.** `CustomEdgeOrchestrator` routes heuristic and RL algorithms through the same event-driven task lifecycle.

(d) **Java–Python environment interface.** `RLEnvServer` turns the Java simulator into a synchronous JSON-over-TCP environment for MAPPO and PPO policies implemented in Python.

(e) **Dual execution modes.** `AbstractRLManager` supports both external training mode and offline inference mode with automatic Python process launch and failure fallback.

(f) **Telemetry and diagnostics.** The simulator exports trajectory CSV files, seed-wise summaries, and RL-specific charts for destination usage, PRB usage, and reward evolution.

2.3 Instantiated Experimental Environment

The previous subsection explains why PureEdgeSim is an appropriate open-source base and why further extensions are required. This subsection records how that simulator is instantiated in this report. The detailed computation, communication, energy, and reward models are formalized later in Section 3; here the emphasis is on the concrete experimental environment used to run the simulations.

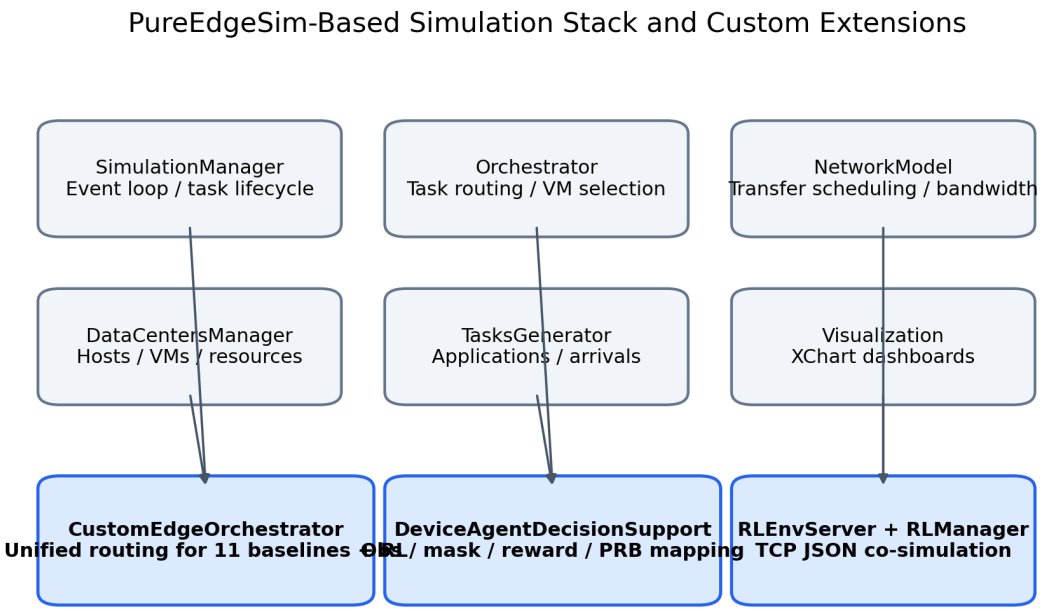


Figure 1: PureEdgeSim modules and the main customization points introduced for this project.

The scenario contains 160 devices in a $200\,\mathrm{m} \times 200\,\mathrm{m}$ area, four edge data centers, and one cloud node. The simulation horizon is 30 minutes, which is deliberately longer than in the Robles PureEdgeSim studies so that each episode generates tens of thousands of scheduling decisions and exposes the learning algorithms to sustained congestion and mobility effects. The overall configuration remains close enough to prior PureEdgeSim practice to preserve comparability, but it is specialized to the LOCAL–EDGE–CLOUD architecture required by this project.

The agent set consists only of devices with `generateTasks=true`. In the current workload, these are the smartphones and sensors. This design follows the same general PureEdgeSim idea used in the Robles papers, namely that learning should be attached to actual task initiators rather than to every entity in the simulator. At the same time, the remaining devices still influence the environment through compute competition, mobility, and shared wireless contention, so the environment stays coupled even though not every entity is an RL agent.

2.4 Event-Driven Task Lifecycle

One of the main reasons for using PureEdgeSim is that it is fundamentally event-driven rather than organized as a fixed-step game loop [11]. The present project keeps that design choice. Each decision is triggered only when a task reaches the orchestration

Parameter	Value
Simulation area	$200\,\mathrm{m} \times 200\,\mathrm{m}$
Number of devices	160
Simulation time	30 min
Mobile device speed	$1.4\,\mathrm{m/s}$
Edge-device communication range	$40\,\mathrm{m}$
Edge data-center coverage radius	$120\,\mathrm{m}$
Cloud equivalent distance	$160\,\mathrm{m}$
WLAN bandwidth	$5000\,\mathrm{Mbps}$
Total PRB blocks	5000
Reference distance d_0	$20\,\mathrm{m}$
Distance exponent α	0.5
Maximum PRBs per task	50 blocks
VM scheduling policy	`SPACE_SHARED`
Random-seed set for final evaluation	9001, 9002, 9003

Table 2: Main simulation parameters used throughout the experiments.

point, not at an artificial global clock tick. This preserves the temporal structure of the simulator and avoids introducing synchronization assumptions that do not exist in the original open-source system.

Once a task is generated, it is sent to the orchestrator, which decides the execution destination and, for RL algorithms, the PRB level used for the wireless upload. After that, the task is transmitted to the selected destination, executed on the assigned VM, and finally returned to the originating device. Fig. 2 shows the high-level flow used in this report.

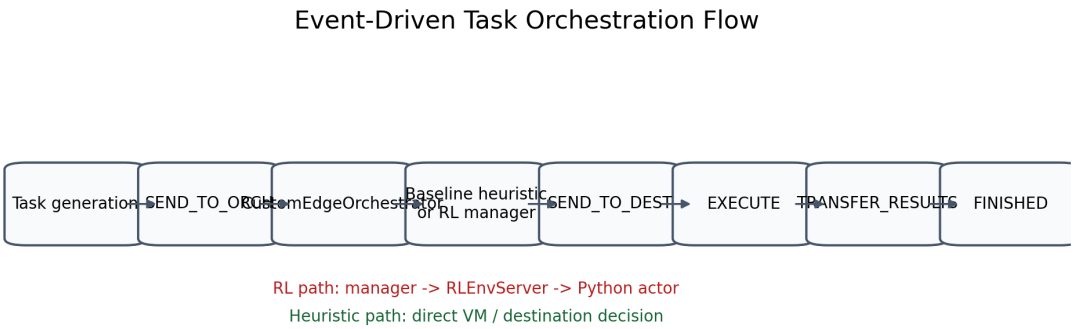


Figure 2: Event-driven task orchestration flow implemented in the customized simulator.

For heuristic baselines, `CustomEdgeOrchestrator` computes the destination directly. For RL-based algorithms, the same orchestrator delegates the decision to `MAPPOManager`

or PPOManager, which collect the observation, query the Python policy, sanitize the returned action, and push the selected destination and PRB allocation back into the event pipeline. This arrangement keeps all algorithms inside the same PureEdgeSim event lifecycle, so every method is evaluated under the same task generator, mobility model, network model, and data-center configuration.

2.5 Java-Python Co-Simulation Architecture

2.5.1 Training Mode

The Robles studies already demonstrated that PureEdgeSim can be modified to host RL-based offloading logic inside the simulator loop [5, 17]. The present project pushes that idea further by separating the simulator and the policy learner across Java and Python. In training mode, Python launches the Java simulator with the environment-server flag enabled. Java creates an RLEnvServer instance and waits for the Python client to connect. The first message is marl_config, which informs Python about the number of agents, observation dimensions, destination labels, and PRB bins. Thereafter, the interaction proceeds in a repeated request-response cycle:

$$\texttt{marl\_turn\_obs} \rightarrow \texttt{marl\_action} \rightarrow \texttt{marl\_transition},$$

followed by marl_episode_end at the end of each episode.

This protocol has two advantages. First, the policy code remains in Python, which simplifies neural-network implementation and logging. Second, the simulator remains authoritative for validity checks, fallback handling, timing, and reward calculation.

2.5.2 Offline Inference Mode

In offline inference mode, Java automatically launches inference_server.py and connects to it through the same TCP protocol. The model path is resolved by priority: an explicitly provided checkpoint is used first, otherwise the latest training run is searched, and finally a legacy fallback path is checked. If no model can be loaded successfully, the manager falls back to a safe default action, ensuring that evaluation runs do not terminate because of a missing checkpoint.

The same message protocol is reused in both modes, so training and evaluation share a common environment interface. This reduces train-test skew and makes the RL pipeline easier to debug.

2.6 Evaluated Scheduling Algorithms

The open-source simulator is used to evaluate not only the proposed MAPPO policy, but also a range of heuristic and learning-based references. This follows the evaluation style used in the Robles papers, where RL methods are judged against greedy or simpler

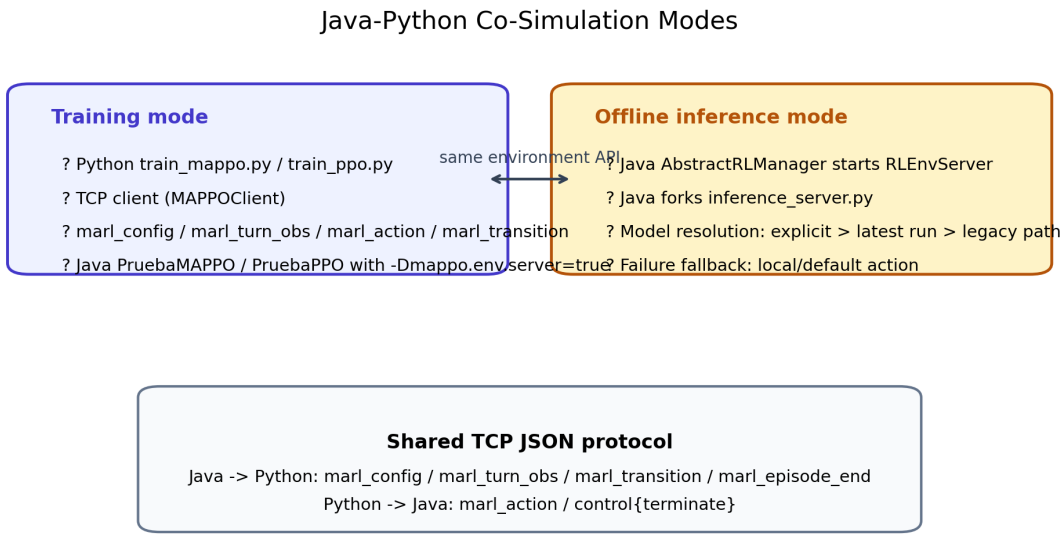


Figure 3: Training-mode and inference-mode data flow in the Java-Python co-simulation architecture.

alternatives rather than in isolation [5, 17]. In this report, MAPPO is compared against ten heuristic schedulers and one single-policy PPO variant. The heuristics span naive, locality-driven, load-driven, and multi-factor strategies so that the evaluation covers both simple and stronger rule-based references.

The PPO baseline is particularly important because it isolates the contribution of the multi-agent design. Both PPO and MAPPO use the same PureEdgeSim-based environment and the same dual decision heads, but MAPPO introduces agent-specific embeddings and centralized training over a global state. This makes the PPO baseline a much closer comparison than a purely heuristic scheduler. Overall, the simulation environment is therefore best understood as an open-source PureEdgeSim foundation, validated in prior literature and then extended in this project toward PRB-aware multi-agent deep reinforcement learning.

Algorithm	Decision rule
RANDOM	Randomly select a feasible VM.
LOCAL	Prefer local execution and fall back to edge or cloud through a simple backup rule.
CLOSEST	Choose the nearest edge node, with queue length as the tie-breaker.
EDGE	Restrict execution to the edge tier and choose the least-loaded feasible VM.
CLOUD	Restrict execution to the cloud tier and choose the least-loaded feasible VM.
ROUND_ROBIN	Select the VM with the smallest task count across all tiers.
TRADE_OFF	Use a weighted cost proportional to $(tasks + 1) \times weight \times taskLength/vmMips$.
INCREASE_LIFETIME	Favor decisions that preserve device battery lifetime.
LATENCY_ENERGY_AWARE	Combine delay and energy terms in a hand-crafted weighted policy.
WEIGHT_GREEDY	Minimize a normalized weighted sum of distance delay, execution delay, load, and energy.
PPO	Single-policy PPO baseline that removes agent embeddings and shares one actor across all devices.

Table 3: Baseline algorithms implemented in the customized orchestrator.

3 System Model and Problem Formulation

3.1 Network Architecture

The target system follows a three-tier LOCAL–EDGE–CLOUD architecture deployed over a $200\,\mathrm{m} \times 200\,\mathrm{m}$ area. All task-generating devices are treated as edge devices in PureEdgeSim terminology, while edge data centers and the cloud server provide additional remote execution capacity. Fig. 4 illustrates the topology used throughout the experiments.

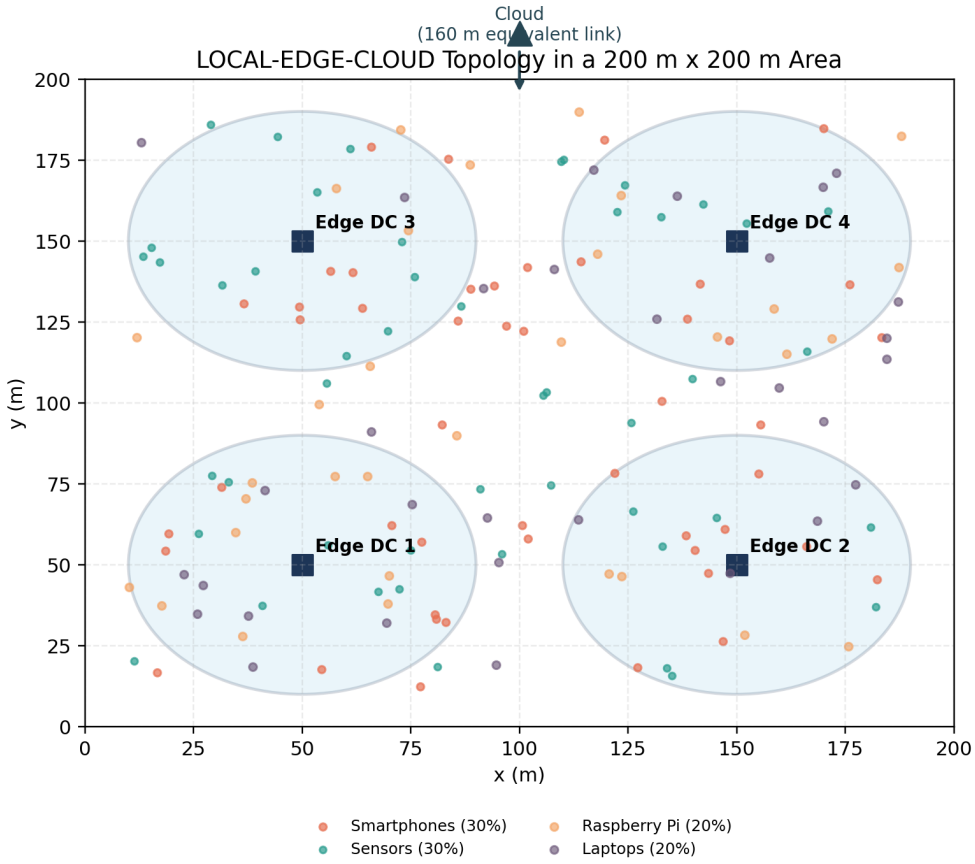


Figure 4: Three-tier LOCAL–EDGE–CLOUD topology used in the experiments.

Four heterogeneous device classes are configured. Smartphones and sensors generate tasks, while Raspberry Pi nodes and laptops act as background devices that contribute mobility, wireless contention, and compute heterogeneity. The edge tier contains four identical edge data centers placed in the four quadrants of the simulation area at $(50,50)$, $(150,50)$, $(50,150)$, and $(150,150)$. Each edge data center contains two hosts, and each host provides eight virtual machines with 60,000 MIPS per VM. The cloud tier contains one data center with two hosts and sixteen VMs in total, each VM provid-

ing 150,000 MIPS. The cloud link is abstracted as a fixed-distance wireless hop with an equivalent distance of 160 m.

Device type	Share	Task gen.	Mobility	CPU	RAM	Local VM
Smartphone	30%	yes	1.4 m/s	20,000 MIPS	4 GB	1 VM
Raspberry Pi	20%	no	fixed	8,000 MIPS	4 GB	1 VM
Laptop	20%	no	fixed	60,000 MIPS	8 GB	1 VM
Sensor	30%	yes	fixed	70,000 MIPS host / no VM	4 GB	none

Table 4: Edge-device configuration extracted from `edge_devices.xml`.

Tier	Count	Placement	VM configuration	Coverage
Edge DC	4	(50, 50), (150, 50), (50, 150), (150, 150)	2 hosts × 8 VMs, 60,000 MIPS/VM	120 m
Cloud	1	abstract remote node	2 hosts × 8 VMs, 150,000 equiv. MIPS/VM	160 m

Table 5: Edge and cloud server configuration.

3.2 Task Model

Three application types are used to generate workloads with different latency and communication characteristics. The rates and workload sizes are defined in `applications.xml`. In every case, a task may be executed locally, offloaded to one of the edge data centers, or sent to the cloud if the communication and resource constraints permit it.

Application	Rate (/min)	Share	Max delay (s)	Request size (KB)	Task length (
AUGMENTED_REALITY	25	20%	8	1500	80,000
E_HEALTH	20	25%	25	10,000	400,000
HEAVY_COMP_APP	25	30%	12	50	120,000

Table 6: Application workload configuration. Result sizes are 100 KB, 10,000 KB, and 50 KB, respectively.

The task lifecycle in PureEdgeSim proceeds through orchestration, transmission, execution, and result return events. In conceptual form, the life cycle can be summarized as

`SEND_TO_ORCH` → `SEND_TO_DEST` → `EXECUTE` → `TRANSFER_RESULTS` → `FINISHED`.

Failures occur when task latency exceeds its deadline, communication becomes infeasible due to mobility or coverage, or no valid execution resource can be allocated.

3.3 Communication Model

Wireless communication is modeled at the PRB level. The WLAN bandwidth is 5000 Mbps, which is discretized into 5000 PRB blocks. If a task is assigned b blocks, the effective bandwidth is given by

$$BW = \frac{BW_{\text{WLAN}}}{PRB_{\text{total}}} \times b \times \min\left(1.0, \left(\frac{d_0}{\max(d, d_0)}\right)^{\alpha}\right), \tag{1}$$

where d is the transfer distance, $d_0 = 20\,\text{m}$ is the reference distance, and $\alpha = 0.5$ is the distance attenuation exponent.

Two PRB allocation modes are supported. For RL-based algorithms, the selected PRB action is converted into a fixed block reservation before transmission begins, and the blocks are released when the upload phase completes. For non-RL algorithms, the requested PRB count is left unset and a dynamic allocator is used. The dynamic allocator builds conflict components over active transfers using breadth-first traversal and then distributes the remaining PRBs according to weighted fairness, where the weight of each transfer is $1.0 + \texttt{lanPriorityBin}$.

Cloud communication uses the same attenuation equation with a fixed equivalent distance of 160 m. This preserves a consistent bandwidth model while abstracting long-haul access into a simulator-friendly wireless cost.

3.4 Computation Model

Virtual machines are scheduled with the `SPACE_SHARED` policy. Let L denote the task length in million instructions and M the MIPS capacity of the assigned VM. The execution time is

$$T_{\text{exec}} = \frac{L}{M}. \tag{2}$$

The destination selector estimates the finish time of every feasible VM and uses this estimate both in the observation features and in the fallback rule applied when an RL action points to an infeasible destination.

3.5 Energy Model

Transmission energy follows the standard radio model

$$E_{\text{tx}} = E_{\text{elec}} \cdot bits + E_{\text{amp}} \cdot bits \cdot d^{n}, \tag{3}$$

where $E_{\text{elec}} = 5 \times 10^{-8}\,\text{J/bit}$, $E_{\text{amp}} = 10^{-11}\,\text{J/bit/m}^2$ in the free-space regime, and $E_{\text{amp}} = 1.3 \times 10^{-15}\,\text{J/bit/m}^4$ in the multipath regime. To stabilize the reward, the total task energy is normalized online through an exponential moving average with smoothing factor $\alpha = 0.01$, which tracks both the running mean and variance of per-task energy consumption.

3.6 Problem Formulation

The objective is to minimize a weighted system cost that reflects deadline satisfaction, execution efficiency, energy consumption, and wireless resource usage. Because the simulator evolves stochastically with many interacting devices, the problem is modeled as a Dec-POMDP

$$\mathscr{G} = \langle \mathscr{N}, \mathscr{S}, \{\mathscr{O}_i\}_{i\in\mathscr{N}}, \{\mathscr{A}_i\}_{i\in\mathscr{N}}, P, R \rangle .$$

Let $\pi = \{\pi_i\}_{i\in\mathscr{N}}$ denote the decentralized device policies. For decision step t, define $F_t \in \{0,1\}$ as the task-failure indicator, $\widetilde{L}_t$ as the normalized latency ratio, $\widetilde{E}_t$ as the normalized energy term, and $\widetilde{B}_t$ as the normalized PRB cost. The long-run optimization target can then be written as

$$J(\pi) = \mathbb{E}_\pi \left[\frac{1}{T} \sum_{t=1}^{T} \left(\lambda_f F_t + \lambda_l \widetilde{L}_t + \lambda_e \widetilde{E}_t + \lambda_b \widetilde{B}_t \right) \right]. \tag{4}$$

In the implemented simulator, this objective is optimized indirectly through the task-level reward

$$R_t = \begin{cases} -5.0, & \text{if the task fails,} \\ 5.0 - 1.0\widetilde{L}_t - 2.0\widetilde{E}_t - 1.5\widetilde{B}_t - P_t^{\text{fb}}, & \text{if the task succeeds,} \end{cases} \tag{5}$$

where $P_t^{\text{fb}} = 1.5$ if the requested destination is replaced by a feasible fallback and 0 otherwise. This reward is a shaped negative version of the cost in Eq. (4). It prioritizes deadline satisfaction first, then penalizes excessive energy consumption, wasteful PRB reservation, unnecessary latency, and policy choices that require environment-side fallback correction.

The agent set $\mathscr{N}$ contains all devices with `generateTasks=true`, ordered by data-center identifier. The global state $\mathscr{S}$ is represented by a 27-dimensional vector that summarizes local observations together with aggregate load, energy, task, and PRB statistics. Each local observation $\mathscr{O}_i$ contains a 13-dimensional device feature vector and an $N_{\text{dest}} \times 11$ destination-feature matrix. The action space is the Cartesian product of a destination choice and an eight-level PRB choice:

$$\mathscr{A}_i = \mathscr{A}_i^{\text{dest}} \times \mathscr{A}_i^{\text{prb}}.$$

The decision process must satisfy the following practical constraints.

- The selected destination must be reachable and admitted by the current network state.
- The assigned VM must exist and be able to complete the task under the simulator rules.
- The global PRB pool must not overflow under fixed RL reservations.
- Task latency must remain within the application deadline to be counted as successful.

State transitions are not modeled analytically; instead, they are induced by the PureEdgeSim event engine. This makes simulation fidelity higher than in a hand-crafted queueing model, while also motivating the use of reinforcement learning instead of closed-form optimization.

4 Methodology

4.1 MAPPO Framework Overview

The proposed method follows the centralized training with decentralized execution (CTDE) paradigm. During training, a centralized critic observes the global simulator state and provides a learning signal that captures system-wide congestion, load balance, and PRB usage. During execution, each device acts independently based only on its own local observation and the feasible-destination mask. This separation is well suited to the target edge-computing scenario because global state is available during simulation and offline training, but real-time deployment should avoid a centralized bottleneck.

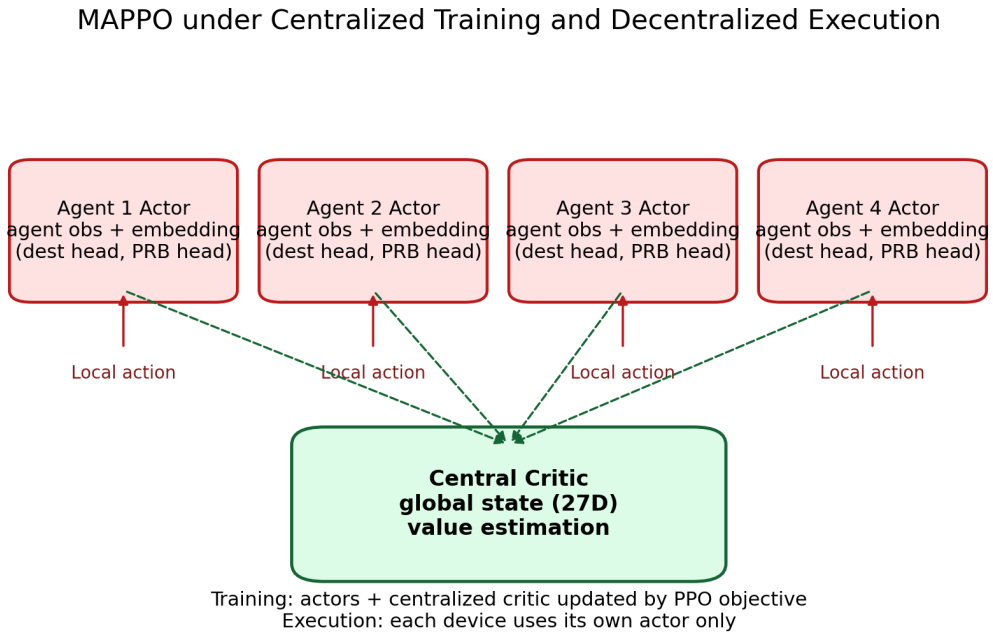


Figure 5: Overall MAPPO framework under centralized training and decentralized execution.

Every task-generating device is treated as one agent. In the current MAPPO version, the multi-agent formulation is strengthened in three concrete ways. First, the actor uses a learnable agent embedding so that different devices do not have to share one undifferentiated latent representation. Second, the centralized critic no longer receives only the static 27-dimensional system summary; it is augmented with recent joint-action telemetry that records how all agents have been choosing destinations and PRB bins over the latest decision window. Third, fallback-aware training is used: destination fallbacks are penalized, and fallback transitions are excluded from the policy-update buffer so that PPO updates remain on-policy. When a new task reaches the orchestrator, the corresponding agent receives its observation, selects an execution destination, and,

if the task is offloaded, selects a PRB allocation level. The environment then applies the action, executes the task in PureEdgeSim, and returns the resulting reward and transition record to the learner.

The design choice of a single dual-head actor contrasts with two alternative decomposition strategies found in prior work. Sun and He [8] use a hierarchical two-level MAPPO in which a high-level agent selects the offloading destination and a separate low-level agent decides the offloading ratio. While this decomposition reduces the action space per agent, it introduces additional complexity in coordinating the two levels and requires careful design of the interface between high-level and low-level policies. By contrast, the dual-head actor used in this report handles both destination and PRB selection within a single forward pass, which simplifies training and avoids the need for hierarchical reward shaping. Similarly, Ke **et al.** [7] use fully decentralized DDQN agents with no centralized critic, which avoids the communication overhead of centralized training but sacrifices the coordination benefits of shared global information. The CTDE paradigm adopted here strikes a balance: agents act independently at execution time (no communication bottleneck), but training exploits a centralized critic that observes system-wide congestion and PRB usage to guide cooperative behavior. This design is particularly well suited to the 160-device scale studied in this report, where explicit coordination through a centralized critic becomes increasingly valuable as the number of interacting agents grows.

4.2 Observation and State Design

The observation design follows the structure implemented in `DeviceAgentDecisionSupport`. For agent i, the local input is

$$o_i = (x_i, D_i, m_i), \tag{6}$$

where $x_i \in \mathbb{R}^{13}$ is the device-level feature vector, $D_i \in \mathbb{R}^{N_d \times 11}$ is the destination-feature matrix, and $m_i \in \{0,1\}^{N_d}$ is the destination-feasibility mask.

The 13-dimensional agent feature vector combines normalized task descriptors, source-device resource indicators, and PRB-availability terms. The destination matrix contains one 11-dimensional feature vector for each candidate destination. The candidates are ordered consistently as local execution first, then edge data centers sorted by data-center identifier, and finally cloud destinations. Each destination feature summarizes its load, compute capacity, estimated finish-over-deadline ratio, distance, network admissibility, destination type, and estimated PRB demand.

The critic uses a two-level state representation. The base global state is a 27-dimensional vector,

$$s_{\text{base}} \in \mathbb{R}^{27}, \tag{7}$$

that aggregates system-wide load indicators, PRB usage, task statistics, and tier-level summaries across the local, edge, and cloud layers. In the updated MAPPO implementation, this base state is augmented with two recent-decision histograms:

$$s_{\text{aug}} = \left[s_{\text{base}}; h^{\text{dest}}; h^{\text{prb}} \right] \in \mathbb{R}^{27+N_d+8}, \tag{8}$$

where $h^{\text{dest}} \in \mathbb{R}^{N_d}$ stores the normalized destination-selection frequencies over the latest 200 decisions and $h^{\text{prb}} \in \mathbb{R}^8$ stores the corresponding PRB-bin frequencies. Under the current topology, $N_d = 6$ because the candidate set contains one local option, four edge data centers, and one cloud destination. Therefore, the actual critic input used by MAPPO in the reported experiments is $27+6+8 = 41$ dimensions. Intuitively, the actor still solves a partially observed local decision problem, while the critic evaluates each action against both the global system state and a compact summary of what the other agents have been doing recently.

To make the feature design explicit, Tab. 7 summarizes the 13 local observation dimensions and Tab. 8 summarizes the destination feature matrix together with the base-state and telemetry components.

Indices	Meaning
0–4	Normalized task length, deadline, request size, result size, and container size.
5–8	Source-device CPU load, source energy level, source running-task count, and best local-VM MIPS.
9–12	Reachable-edge count, remaining PRB ratio, normalized simulation time, and normalized affordable PRB ceiling.

Table 7: Agent-observation dimensions implemented in `DeviceAgentDecisionSupport`.

The destination mask plays a central role in maintaining valid actions. If a destination is out of coverage, lacks a feasible VM, or violates offloading constraints, its mask value is set to zero and the corresponding action logit is suppressed before sampling. This masked policy is more sample efficient than allowing invalid actions and penalizing them afterward.

4.3 Action Space Design

The agent action is factorized into a destination head and a PRB head:

$$a_i = \left(a_i^{\text{dest}}, a_i^{\text{prb}} \right). \tag{9}$$

The destination head selects one element from the ordered candidate list. The PRB head selects one out of eight discrete bins. The bins represent relative fractions of the per-task PRB limit:

$$\rho \in \{0.02, 0.05, 0.10, 0.20, 0.40, 0.60, 0.80, 1.00\}. \tag{10}$$

Component	Meaning
Destination 0–2	Destination CPU load, running-task count, and total MIPS.
Destination 3–5	Estimated finish-over-deadline ratio, normalized distance, and network-admissibility flag.
Destination 6–10	Local / edge / cloud type flags, VM-count summary, and estimated PRB need.
Base state 0–12	Direct copy of the active agent's 13-dimensional local observation.
Base state 13–26	Source CPU mean/max, source energy mean/max, source running-task mean/max, edge CPU mean/std, edge running-task mean/std, cloud CPU mean, active-task ratio, allocated-PRB ratio, and destination CPU-imbalance std.
Telemetry 27–$26+N_d$	Normalized histogram of recent destination selections over the last 200 MAPPO decisions.
Telemetry $27+N_d$–$34+N_d$	Normalized histogram of recent PRB-bin selections over the same 200-decision window.

Table 8: Destination-feature groups and critic-state composition.

Given the simulator setting `maxPerTask` $= 50$, the selected ratio is converted into an integer reservation capped at 50 blocks.

If the chosen destination is local execution, the PRB action is ignored and replaced by zero because no wireless upload is required. In the implementation, the PRB log-probability and entropy contributions are also neutralized for such local actions so that the policy is not penalized for an irrelevant bandwidth choice.

Although destination masking prevents most invalid actions, a final destination fallback rule is retained for robustness. If the sampled action still points to an infeasible destination at execution time, the system falls back to the feasible destination with the shortest estimated completion time. In the final MAPPO evaluation runs used in this report, the fallback count is zero across all three seeds, indicating that the trained policy learns to stay within the valid action region.

4.4 Network Architecture

4.4.1 TurnActor (MAPPO Actor)

The MAPPO actor is implemented as a `TurnActor`. Each agent identifier is mapped to a learnable 16-dimensional embedding e_i. The embedding is concatenated with the 13-dimensional agent observation and passed through a two-layer multilayer perceptron with hidden size 128 and tanh activations:

$$h_i = f_\theta^{\text{agent}}([x_i; e_i]). \tag{11}$$

The embedding table can be resized dynamically when the number of task-generating devices changes, which avoids hard-coding the agent count into the model definition.

Each candidate destination feature vector $d_{i,j}$ is concatenated with the repeated agent hidden state and encoded by a destination-specific multilayer perceptron:

$$u_{i,j} = f_\theta^{\text{dest}}([d_{i,j}; h_i]), \tag{12}$$

again using a two-layer MLP with hidden size 128 and tanh activations. A linear destination head then scores every encoded candidate:

$$\ell_{i,j}^{\text{dest}} = w_{\text{dest}}^\top u_{i,j} + b_{\text{dest}}, \tag{13}$$

followed by masked softmax and categorical sampling:

$$\pi_\theta^{\text{dest}}(j \mid o_i) = \text{Softmax}\Big(\text{Mask}\Big(\ell_{i,j}^{\text{dest}}, m_i\Big)\Big). \tag{14}$$

The PRB head is conditioned on the selected destination. In the implementation, the hidden vector of the selected destination is concatenated with the agent hidden state and passed to a dedicated PRB head:

$$\ell_i^{\text{prb}} = g_\theta\left([h_i; u_{i,a_i^{\text{dest}}}]\right), \tag{15}$$

followed by

$$\pi_\theta^{\text{prb}}(k \mid o_i, a_i^{\text{dest}}) = \text{Softmax}\Big(\ell_i^{\text{prb}}\Big). \tag{16}$$

This coupling is important because the desirable PRB level differs substantially between nearby edge nodes and the farther cloud tier. In code, the PRB head is realized as `Linear(256,128)` $\rightarrow$ tanh $\rightarrow$ `Linear(128,8)`.

4.4.2 CentralCritic

The centralized critic takes the augmented global state as input and outputs one scalar value estimate:

$$V_\phi(s_{\text{aug}}) = f_\phi^{\text{critic}}(s_{\text{aug}}). \tag{17}$$

The critic uses a two-layer MLP with hidden size 256 and tanh activations. For the reported 6-destination scenario, the input dimension is therefore 41 rather than 27. This change is central to the updated MAPPO design: the critic no longer evaluates a device action only against static load and PRB summaries, but also against recent joint behavior of the full agent population. Because the critic sees aggregated system information that is unavailable to an individual device, it can provide a more coordination-aware baseline for policy optimization than a purely local value function.

4.4.3 PPO Baseline Actor

The PPO baseline shares the same destination encoder, PRB head, and overall dual-head logic, but removes the learnable agent embedding. Its actor receives only the raw 13-dimensional device observation before constructing the same destination-conditioned hidden states. In addition, PPO keeps the original 27-dimensional critic state instead of the telemetry-augmented 41-dimensional MAPPO state. As a result, PPO must represent all device roles through one shared feature space and one shared value context, whereas MAPPO can condition policy evaluation on both explicit agent identity and recent multi-agent coordination statistics. These are the main controlled distinctions between the two RL methods.

4.5 Reward Function Design

The reward is defined at the task level:

$$R = \begin{cases} -5.0, & \text{if the task fails,} \\ 5.0 - 1.0 \cdot latencyRatio - 2.0 \cdot energyNorm - 1.5 \cdot networkCost - fallbackPenalty, & \text{if the task succeed} \end{cases} \tag{18}$$

Here,

$$latencyRatio = \frac{\text{actual latency}}{\text{maximum allowed latency}}, \tag{19}$$

$$energyNorm = \frac{E_t - \mu_t}{\sqrt{\sigma_t^2 + \varepsilon}}, \tag{20}$$

$$networkCost = \frac{\text{reserved PRB blocks}}{50}, \tag{21}$$

and

$$fallbackPenalty = \begin{cases} 1.5, & \text{if the requested destination is replaced by a feasible fallback,} \\ 0, & \text{otherwise.} \end{cases} \tag{22}$$

This formulation explicitly favors deadline satisfaction while still discouraging wasteful wireless reservations, excessive transmission energy, and policy choices that require environment-side fallback. A failed task receives a fixed negative reward because lateness, mobility loss, or network failure all indicate a clear scheduling breakdown. For successful tasks, the positive base reward is gradually reduced according to delay, energy, PRB cost, and fallback. Compared with the earlier P95-normalized energy penalty, the current version uses online standardization through an exponential moving average with $\alpha = 0.01$, which makes unusually energy-hungry decisions more visible to the learner.

4.6 Training Algorithm

Policy optimization follows the clipped PPO objective [20]. For trajectory sample t,

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right], \tag{23}$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}. \tag{24}$$

For the finalized training artifact used in this report, the implementation adopts a one-step event-level advantage rather than a long-horizon bootstrapped estimator. Specifically, the update code uses

$$R_t = r_t, \tag{25}$$

$$\hat{A}_t = r_t - V_\phi(s_t), \tag{26}$$

followed by minibatch normalization of $\hat{A}_t$. This choice matches the event-driven nature of the environment: each transition already corresponds to a largely self-contained offloading transaction whose reward summarizes latency, energy, and PRB usage after the task outcome is known. The shared Python utilities still contain a generic GAE helper for future extensions, but the MAPPO and PPO runs reported here use the immediate-return form above.

One additional implementation detail is critical for the updated MAPPO stability. If the environment replaces the requested destination with a feasible fallback, that transition is not inserted into the MAPPO update buffer. Otherwise, the PPO ratio would be computed against an action that the policy did not actually sample. For non-fallback transitions, the training code recomputes the log-probability using the original Python action rather than the post-resolution executed action. Together with the mask-aware default destination on timeout and the longer action timeouts used during training and offline inference, this removes an important source of off-policy noise that affected the earlier implementation.

Entropy regularization is linearly annealed from 0.02 to 0.002, encouraging exploration early in training and sharper decisions later. The main hyperparameters are listed in Tab. 9. Several of these choices are informed by the systematic study of Yu **et al.** [15], which identifies five critical implementation factors for PPO in cooperative multi-agent settings. In particular, the clipping range $\varepsilon = 0.1$ stays within their recommended bound of $\varepsilon < 0.2$; the number of PPO epochs per update (4) falls within their suggested range of 5–10 for hard tasks; and the centralized critic input combines both global system features and agent-specific local information, following their finding that the AS (agent-specific global state) representation consistently outperforms alternatives that use only local observations or only environment-provided global state.

Hyperparameter	Value
Training episodes	40
PPO clip ε	0.1
Actor learning rate	3×10^{-4}
Critic learning rate	3×10^{-4}
PPO epochs per update	4
Minibatch size	1024
Episodes per update	1
Entropy coefficient	$0.02 \rightarrow 0.002$
Max gradient norm	0.5
Agent embedding dimension	16
Actor hidden dimension	128
Critic hidden dimension	256
PRB bins	8

Table 9: Main training hyperparameters used for MAPPO and the PPO baseline.

The runtime configuration still records a nominal $\gamma = 0.99$ for compatibility with the shared training scripts, but the reported runs do not bootstrap across future rewards in the update rule above.

Algorithmically, one training episode proceeds as follows.

1. Java starts a 30-minute PureEdgeSim episode and exposes the MARL interface through `RLEnvServer`.
2. Every time a task reaches the orchestrator, the active device sends a masked local observation to Python.
3. Python samples a destination and PRB action, sends the decision back, and stores the resulting transition.
4. At the end of the episode, the centralized critic computes values, advantages are estimated, and PPO updates the actor and critic for four epochs.
5. The best-performing checkpoints are then reused for offline inference and multi-seed evaluation.

The resulting method preserves decentralized execution at deployment time while still exploiting centralized information during training, which is the key reason MAPPO is attractive for large, interacting edge-computing systems.

5 Experiment Results

This chapter presents the experimental evaluation of the proposed MAPPO-based joint offloading and PRB allocation framework described in Section 4. The experiments are organized into five groups: (i) the standard 40-episode MAPPO training and evaluation run, (ii) a device-count scalability study examining MAPPO performance under 80, 160, 240, and 320 devices, (iii) a MAPPO ablation study isolating the contributions of agent embeddings and adaptive PRB control, (iv) a 10-algorithm offline comparison under the same 160-device scenario, and (v) a direct MAPPO–PPO comparison.

5.1 MAPPO under the Standard 40-Episode, 30-Minute Setting

The standard MAPPO experiment uses the default configuration introduced in Section 2: a 30-minute simulation horizon, 160 task-generating devices, four edge data centers, one cloud node, and the LOCAL–EDGE–CLOUD architecture. The training run contains 40 episodes, each executed as a full PureEdgeSim episode driven by the Java–Python co-simulation loop.

Training behavior. The updated MAPPO learns a strong policy under this standard setting. The task success rate increases from 57.85% in episode 1 to 99.91% in episode 40. The mean success rate improves from 75.79% over the first five episodes to 99.87% over the last five episodes. MAPPO crosses 95% success by episode 6 and 99% success by episode 13, while the best episode is the final checkpoint itself at 99.91%. At the same time, the average total task time drops from 5.0108 s to 4.6622 s, and the number of delay-induced failures decreases from 28,193 to only 52. These trends indicate genuine policy improvement rather than a trivial single-tier shortcut.

The full training trajectory is shown in Fig. 6. The most important observation is not only the high final success rate, but also the stable late-stage plateau after the multi-agent telemetry update and fallback-aware training fix. The mean success rate over the last 10 episodes is 99.80% with a standard deviation of 0.131. The training log records an average simulator wall time of 227.74 s per episode, or 9109.46 s in total for the 40 episodes, which is about 2.53 hours end to end.

Episode	Success (%)	Avg. Total Time (s)	Energy (W)	Delay Failures
1	57.85	5.0108	187.1040	28193
5	91.96	5.0888	221.2397	5313
10	97.90	4.7186	244.1243	1382
20	99.62	4.5733	246.1816	244
30	99.19	4.6681	234.5059	538
40	99.91	4.6622	226.7990	52

Table 10: Selected MAPPO checkpoints under the standard training setting.

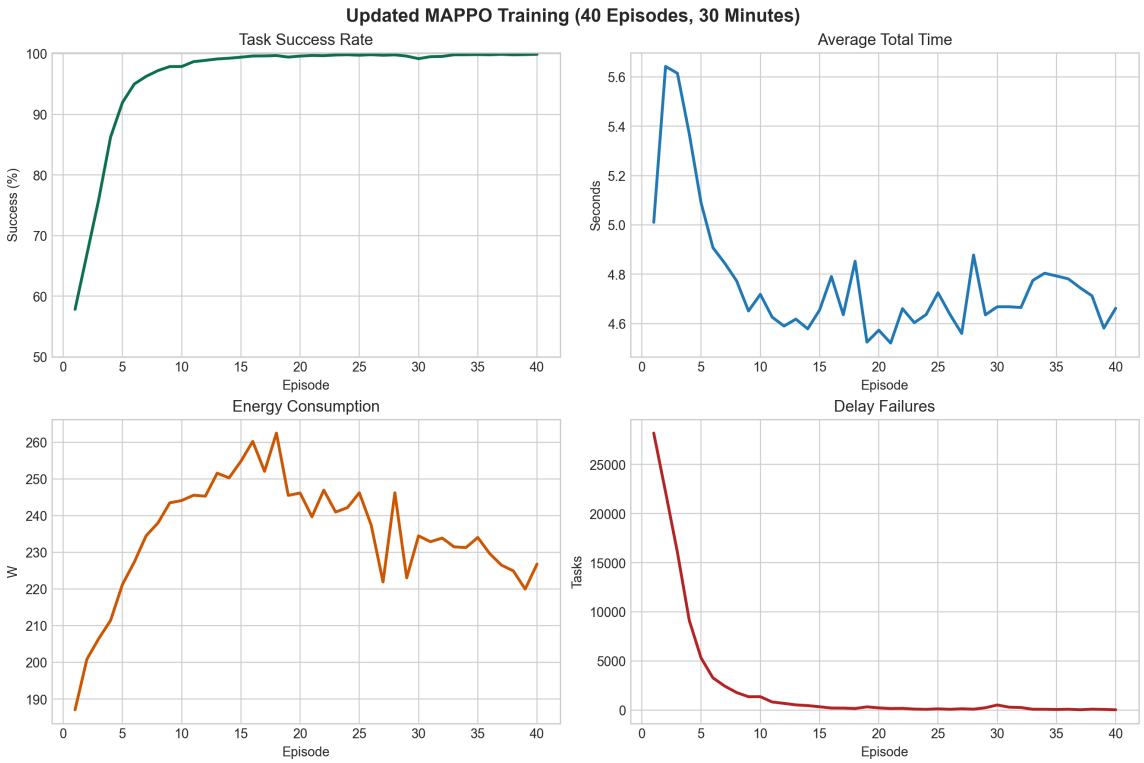


Figure 6: MAPPO training progress under the updated multi-agent setting.

Evaluation behavior across seeds. The final MAPPO checkpoint was evaluated on three seeds: 9001, 9002, and 9003. In contrast to the earlier preserved artifacts, the updated MAPPO is now stable on all three seeds. The success rates are 99.8901%, 99.9390%, and 99.9421%, respectively. The three-seed mean success rate is therefore 99.9237% with a standard deviation of only 0.0238. The mean average total time is 4.5284 s, the mean energy consumption is 227.6360 W, and the fallback rate is exactly zero on every seed.

Seed	Success (%)	Avg. Total Time (s)	Energy (W)	Fallback Rate (%)
9001	99.8901	4.6031	233.0438	0.00
9002	99.9390	4.4367	220.7694	0.00
9003	99.9421	4.5454	229.0948	0.00
Mean	99.9237	4.5284	227.6360	0.00

Table 11: MAPPO evaluation results across three test seeds.

Fig. 7 makes the new robustness explicit. All three seeds remain tightly clustered in success rate, average total time, and energy consumption. Most importantly, seed 9003 no longer exhibits the catastrophic collapse seen in the older checkpoint. The updated MAPPO therefore removes the previous robustness objection and makes the multi-agent design viable for the preserved evaluation setting.

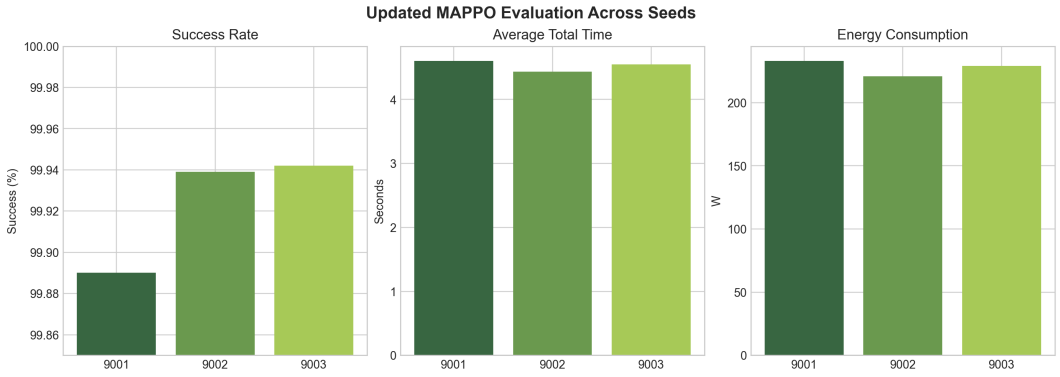


Figure 7: Seed-wise evaluation of the updated MAPPO checkpoint across three test seeds.

Behavioral analysis. The action statistics stored in the MAPPO trajectory analysis show that the policy learned a meaningful joint scheduling pattern. On seed 9001, MAPPO sends 51.73% of decisions to the cloud, 19.81% to local execution, and the remaining 28.46% to the four edge data centers. The PRB policy remains concentrated on the 20%, 80%, and 100% bins, with the 20% bin clearly dominant at 72.76%. This indicates that the policy is not simply overusing one destination or one PRB level, but is instead coordinating compute-tier choice and wireless reservation.

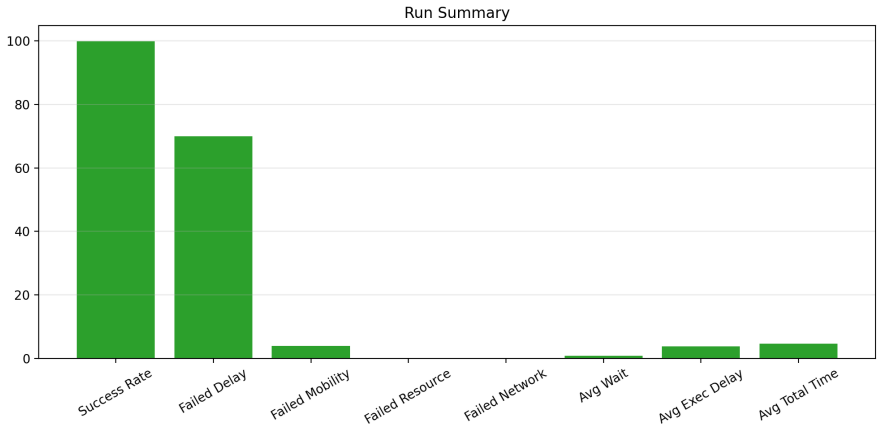
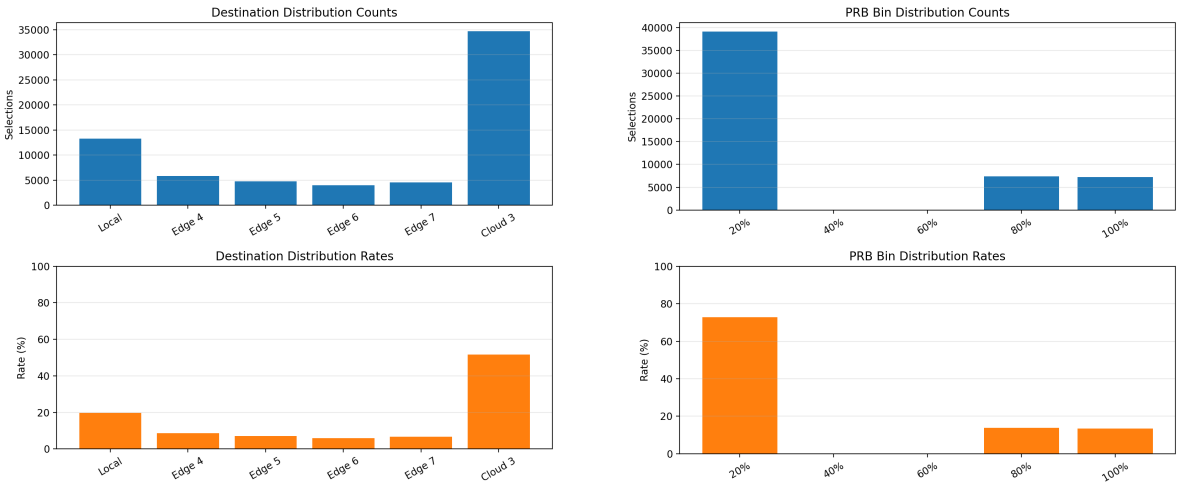


Figure 8: Overall MAPPO evaluation summary for the representative standard seed 9001.



(a) Destination distribution on seed 9001.

(b) PRB-bin distribution on seed 9001.

Figure 9: Detailed MAPPO action behavior for the stable standard evaluation seed.

More importantly, the same pattern remains stable across all three seeds. The cloud-selection rate stays within 51.10% to 52.37%, the local-selection rate stays within 19.09% to 20.51%, and the 20% PRB bin remains dominant at roughly 72% to 73%. Seed 9002 shifts slightly more traffic toward Edge 7 and the 100% PRB bin, while seed 9003 shifts slightly more traffic toward Edge 5, but these are minor policy adjustments rather than instability. Fig. 10 shows that even on seed 9003 the selection frequencies remain aligned with destination availability, while fallback count and network failures remain zero.

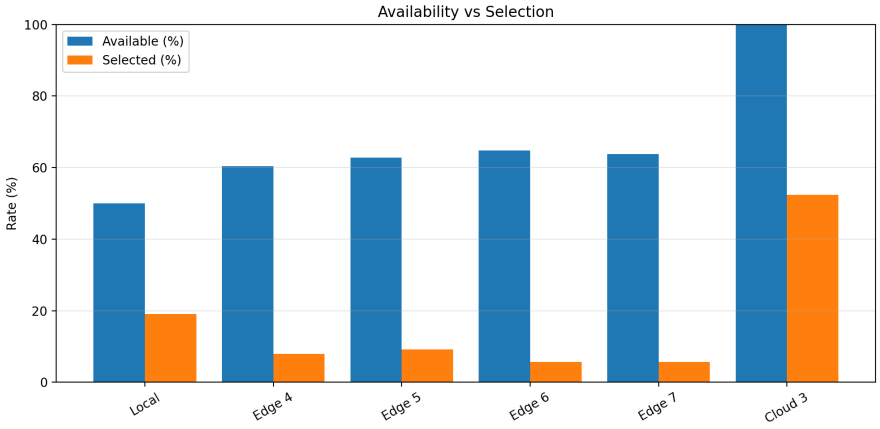
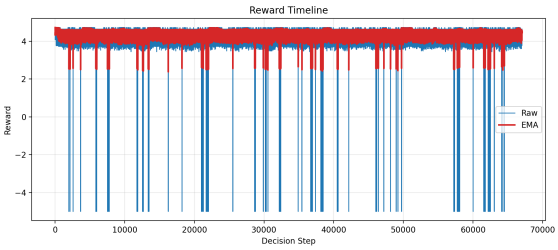


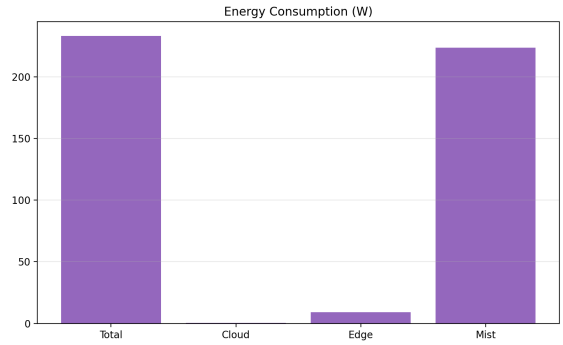
Figure 10: Availability versus actual selection for MAPPO on seed 9003 after the update.

Per-decision reward and energy analysis. The remaining seed-level charts explain why the updated MAPPO remains stable once training converges. On the representative seed 9001, the raw reward trajectory in Fig. 11 is concentrated near the success regime, while the exponential moving average stays mostly between 4.0 and 4.6. The occasional negative spikes correspond to the rare delay or mobility failures already quantified in Tab. 11, but they are sparse enough that they do not perturb the long-run policy. In other words, the learned policy is not only successful on average; it is locally stable over tens of thousands of sequential offloading decisions.

The energy breakdown in the same figure clarifies where the cost is paid. For seed 9001, total energy consumption is 233.0438 W, of which 223.4854 W comes from mist/local execution, 8.9694 W from edge servers, and only 0.5890 W from the cloud. This means the energy metric is still dominated by the battery-powered device layer even though more than half of the tasks are offloaded to the cloud. The practical implication is that cloud offloading mainly relieves deadline pressure, whereas overall energy efficiency is still governed by how many tasks remain on local devices and how aggressively uplink PRBs are reserved.



(a) Reward timeline on the representative evaluation seed 9001.



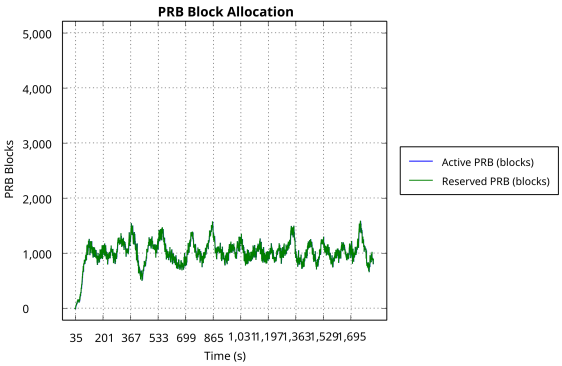
(b) Energy breakdown on the representative evaluation seed 9001.

Figure 11: Per-decision reward stability and energy decomposition for the updated MAPPO policy under the standard setting.

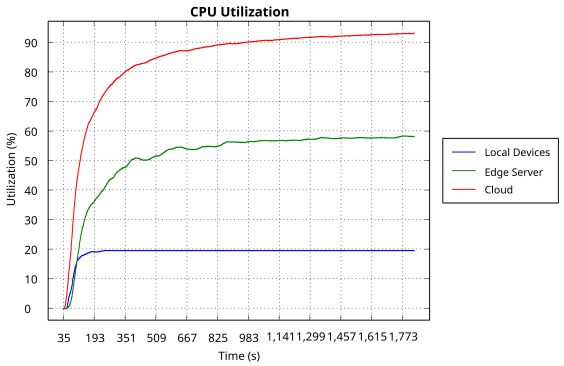
Runtime resource dynamics. The simulator-side runtime charts provide a complementary view of how the learned policy interacts with the physical resource layer over the full 30-minute horizon. Fig. 12 shows the PRB block allocation timeline and the CPU utilization breakdown for the representative seed 9001. The PRB chart reveals that the policy maintains a moderate and stable wireless reservation throughout the episode, with occasional short bursts when multiple offloading decisions coincide. This is consistent with the 20% PRB-bin dominance observed in the action statistics: the policy avoids saturating the shared wireless pool while still reserving enough capacity for timely uplink transfers. The CPU utilization chart shows that cloud and edge servers are consistently loaded at moderate levels, while mist-layer utilization remains low, confirming that the policy successfully distributes compute pressure across tiers rather than overloading any single layer.

The delay distribution and task failure timeline in Fig. 13 further confirm the policy's operational stability. The delay chart shows that the vast majority of tasks complete well within their deadlines, with only a thin tail of near-deadline completions. The task failure timeline remains essentially flat at zero throughout the episode, with the 52 delay failures from Tab. 10 spread so thinly across the 30-minute window that they are visually indistinguishable from zero. This level of temporal stability is important because it rules out the possibility that the high aggregate success rate masks periodic failure bursts during congestion peaks.

Overall, the standard-setting MAPPO results are now much stronger than before. MAPPO clearly learns a high-quality joint offloading and PRB policy, reaches near-perfect operation by the end of training, and no longer exhibits the severe seed sensitivity that previously weakened the case for explicit multi-agent modeling.

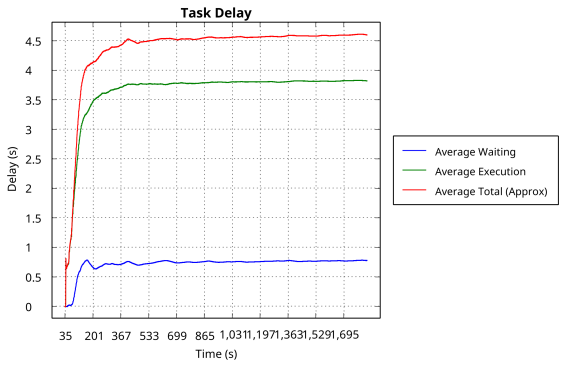


(a) PRB block allocation over time on seed 9001.

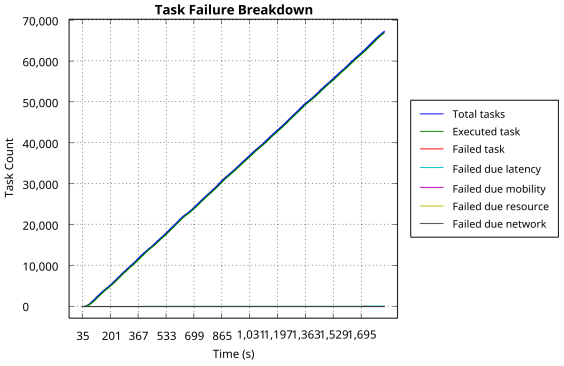


(b) CPU utilization breakdown on seed 9001.

Figure 12: Runtime resource dynamics for the MAPPO policy under the standard evaluation setting (seed 9001).



(a) Delay distribution on seed 9001.



(b) Task failure timeline on seed 9001.

Figure 13: Delay distribution and task failure timeline for the MAPPO policy under the standard evaluation setting (seed 9001).

5.2 Performance of MAPPO under Different Numbers of Devices

To evaluate the scalability of the proposed MAPPO framework, a device-count ablation study was conducted under the same 40-episode training setting, 30-minute simulation horizon, four edge data centers, and one cloud node. The only variable is the number of task-generating edge devices, which is swept over $\{80, 160, 240, 320\}$. Each configuration is trained independently from scratch and then evaluated on three seeds (9001, 9002, 9003). The infrastructure topology and all hyperparameters remain identical across configurations.

Training convergence. Fig. 14 shows the episode reward curves for all four device counts. The 80-device and 160-device configurations converge quickly and stably: the 80-device model plateaus around episode 15 at approximately 150 K reward, while the 160-device model reaches its plateau near 292 K by episode 17. The 240-device configuration exhibits a more turbulent learning trajectory, starting from deeply negative rewards (-233 K in episode 1) and spending the first 10 episodes in a volatile exploration phase before eventually converging to approximately 420 K by episode 22. This suggests that 40 episodes are barely sufficient for 240 devices. The 320-device configuration, by contrast, fails to converge entirely: its reward oscillates between -35 K and $+39$ K throughout all 40 episodes, never establishing a stable positive trend.

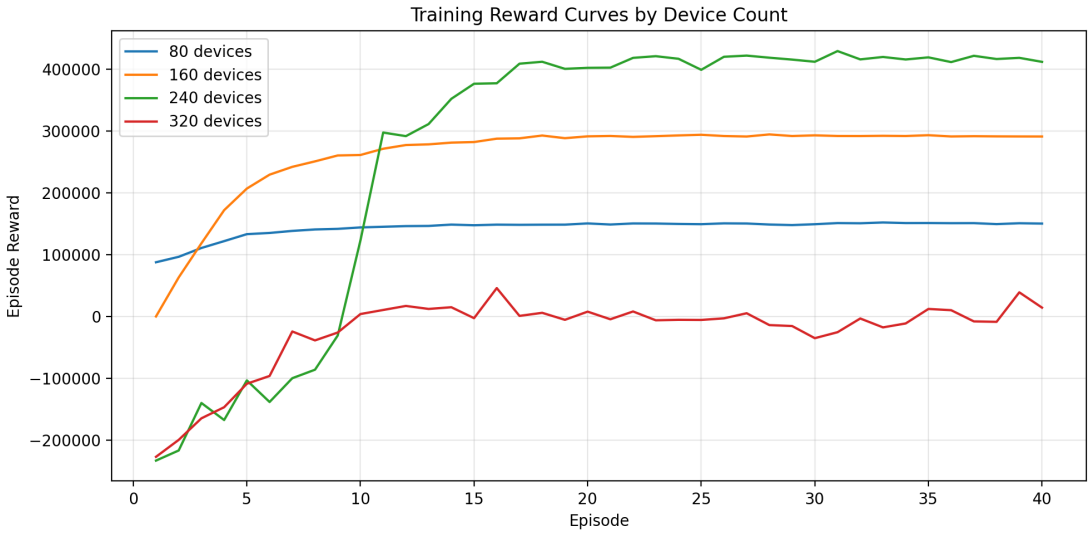


Figure 14: Training reward curves for MAPPO under different device counts. The 80, 160, and 240-device configurations converge within 40 episodes, while the 320-device configuration fails to converge.

Evaluation performance. Tab. 12 summarizes the three-seed evaluation metrics for each device count. The results reveal a clear two-regime pattern: MAPPO scales grace-

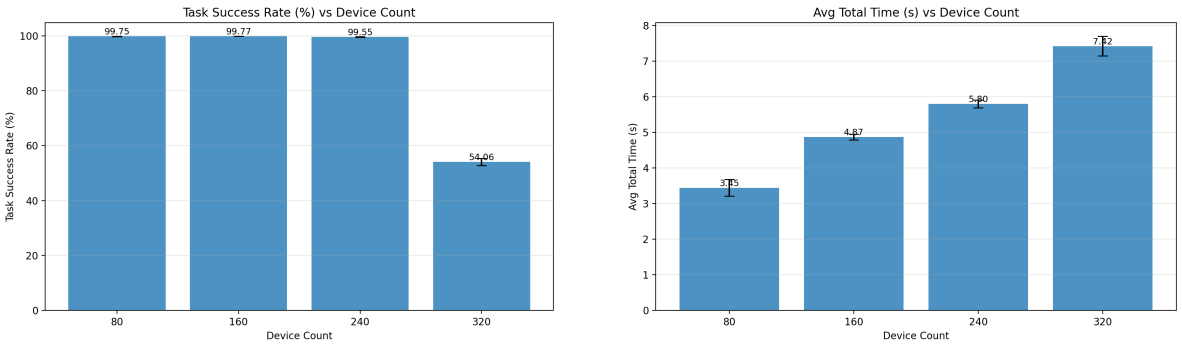
fully from 80 to 240 devices, maintaining task success rates above 99.4%, but suffers a catastrophic performance collapse at 320 devices.

Devices	Decisions	Success (%)	Time (s)	Energy (W)	Delay Fail	Reward
80	33 639	99.75	3.45	111.24	86	151 057
160	67 016	99.77	4.87	227.76	154	290 588
240	100 546	99.55	5.80	458.78	451	433 995
320	130 744	54.06	7.42	626.30	61 899	−5 192

Table 12: Three-seed mean evaluation metrics for MAPPO under different device counts. Fallback rate is zero for all configurations. All values are averaged over seeds 9001, 9002, and 9003.

From 80 to 240 devices, the per-agent reward remains consistently positive and the success rate stays above 99.4%, indicating that MAPPO effectively coordinates an increasing number of agents without degrading individual decision quality. The total reward scales roughly in proportion to the number of decisions, confirming that each additional agent contributes positively. Average total time increases moderately from 3.45 s to 5.80 s, which is expected as more agents compete for shared edge and cloud resources. Energy consumption also scales proportionally with device count, reflecting the additional mist-layer power draw from more active devices.

Fig. 15 visualizes the task success rate and average total time across device counts. The success rate remains flat near 100% for the first three configurations before dropping sharply at 320 devices, while the average total time increases gradually and then jumps at 320 devices.



(a) Task success rate across device counts.

(b) Average total time across device counts.

Figure 15: Task success rate and average total time for MAPPO under different device counts. Performance remains stable from 80 to 240 devices but collapses at 320 devices.

The 320-device collapse: infrastructure saturation. The 320-device failure is not caused by insufficient training or a limitation of the MAPPO algorithm itself. Rather, it

reflects a fundamental infrastructure capacity bottleneck. At 320 devices, the simulator generates approximately 130 K task decisions per episode, a 30% increase over the 240-device case. However, the number of delay failures jumps from 451 to 61,899—a 137× increase—indicating that the system has crossed a resource saturation threshold.

The evidence for infrastructure saturation is visible in the learned policy's behavioral adaptation. Tab. 13 shows how the destination distribution shifts as device count increases. At 80 devices, MAPPO sends roughly 60% of tasks to the cloud, leveraging its stable bandwidth and ample compute. As device count grows, the cloud fraction steadily decreases: from 60% at 80 devices to 52% at 160, 31% at 240, and only 12% at 320. Simultaneously, the local execution fraction rises from 20% to 51%. This shift demonstrates that the policy is actively adapting to cloud congestion by redirecting traffic locally. At 320 devices, however, even with half of all tasks executed locally, the remaining offloaded workload still overwhelms the edge and cloud tiers, resulting in massive queuing delays. The `average_real_total_time_s` metric—which includes queuing time—reaches 26–36 s at 320 devices, compared with 3.2 s at 80 devices, confirming severe resource contention.

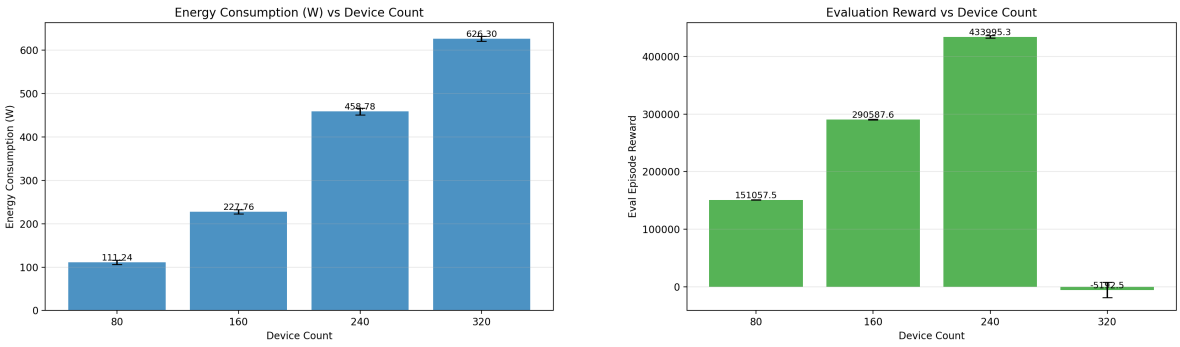
Devices	Local (%)	Edge Total (%)	Cloud (%)
80	20.6	19.7	59.6
160	19.8	27.9	52.1
240	41.0	28.2	30.8
320	50.7	37.5	11.7

Table 13: Three-seed mean destination distribution for MAPPO under different device counts. The policy progressively shifts from cloud-dominant to local-dominant as infrastructure becomes saturated.

Fig. 16 shows the energy consumption and evaluation reward across device counts. Energy scales roughly linearly from 80 to 320 devices, reflecting the proportional increase in mist-layer power draw. The evaluation reward, however, turns sharply negative at 320 devices, confirming that the policy cannot generate net positive value when infrastructure capacity is exhausted.

Fig. 17 further illustrates the saturation effect. The number of delay-induced failures remains negligible from 80 to 240 devices but explodes at 320 devices, forming a clear inflection point that marks the infrastructure capacity boundary.

Summary. The device-count ablation reveals that MAPPO scales effectively within the capacity envelope of the underlying infrastructure. From 80 to 240 devices, the policy maintains near-perfect task success rates while adaptively shifting its destination distribution to balance load across tiers. The 320-device collapse is not a failure of multi-agent learning but a consequence of fixed infrastructure capacity: four edge data centers and one cloud node cannot absorb the workload generated by 320 concurrent



(a) Energy consumption across device counts.

(b) Evaluation reward across device counts.

Figure 16: Energy consumption and evaluation reward for MAPPO under different device counts.

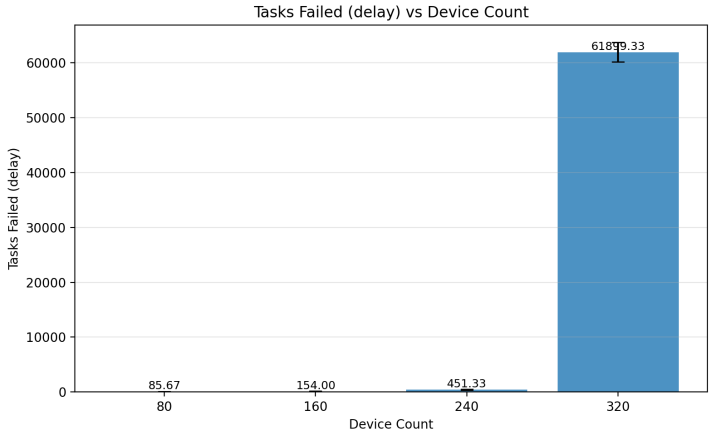


Figure 17: Delay-induced task failures across device counts. The sharp jump at 320 devices indicates infrastructure saturation rather than policy degradation.

devices within deadline constraints. Supporting larger device populations would require expanding the infrastructure—adding edge data centers, increasing per-node VM capacity, or both—rather than modifying the learning algorithm.

5.3 Ablation Study of MAPPO

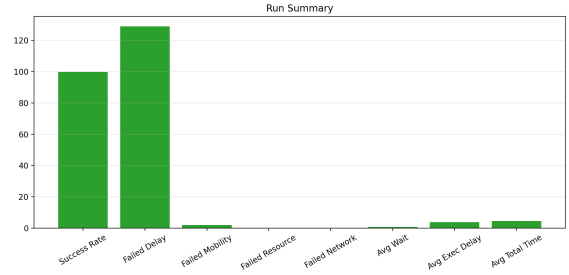
Two controlled ablations were conducted under the same 40-episode, 30-minute, 160-device setting as the full MAPPO run. **NoEmbedding** removes the 16-dimensional agent embedding while keeping the same dual-head actor and centralized critic. **FixedPRB** keeps the offloading policy but forces every non-local transfer to use the same 80% PRB request, thereby disabling adaptive radio allocation. These two variants isolate two different questions: whether explicit agent identity matters, and whether adaptive PRB control matters.

Variant	Mean Success (%)	Std. Success	Mean Total Time (s)	Mean Energy (W)	Mean Delay Fai
MAPPO	99.9237	0.0238	4.5284	227.6360	48.00
NoEmbedding	99.8148	0.0245	4.4991	227.7301	123.00
FixedPRB	92.3822	1.0314	4.7047	309.2645	5127.67

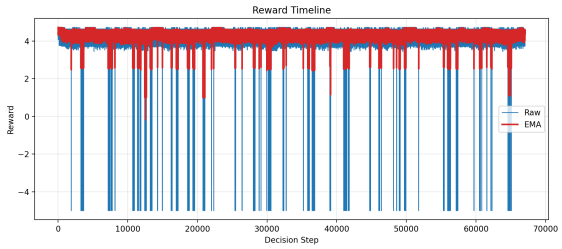
Table 14: Three-seed evaluation summary of the MAPPO ablations.

NoEmbedding: controlled but measurable degradation. The NoEmbedding model still converges to a strong scheduler, but it is consistently weaker than the full MAPPO policy in reliability. During training, success rises from 57.44% in episode 1 to 99.69% in episode 40, which is only slightly below the full model's 99.91%. The difference appears mainly in the tail of the failure distribution rather than in coarse routing behavior: the three-seed mean success drops from 99.9237% to 99.8148%, while mean delay failures increase from 48 to 123. The mean total time is slightly lower at 4.4991 s, but this marginal latency gain does not compensate for the worse deadline compliance. The mean simulator wall time also drops from 227.74 s per episode to 202.86 s, so the ablation is cheaper to train, but not better in the metric that matters most for the scheduler.

The representative seed-9001 charts in Fig. 18 and Fig. 19 show why the degradation is modest rather than catastrophic. The run-summary chart is still dominated by the success bar, and the reward EMA remains high and positive across most of the episode. However, the negative reward spikes are more frequent and the EMA dips more noticeably than in the full MAPPO case, indicating less stable local decision quality. The destination-distribution chart remains almost unchanged, with 51.78% of tasks still sent to the cloud and 19.83% kept local. The main behavioral shift appears in the PRB chart: compared with the full MAPPO policy, NoEmbedding moves probability mass from the 80% bin to the 100% bin, with the latter rising to 16.94%. This suggests that once agent identity is removed, the shared policy compensates by using slightly more aggressive uplink reservation on difficult decisions.

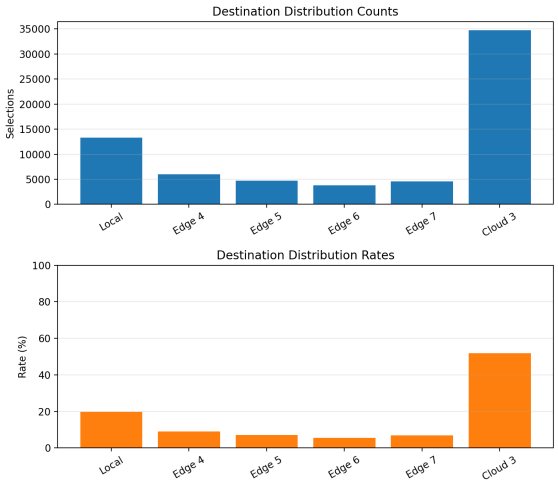


(a) Representative seed-9001 run summary for NoEmbedding.

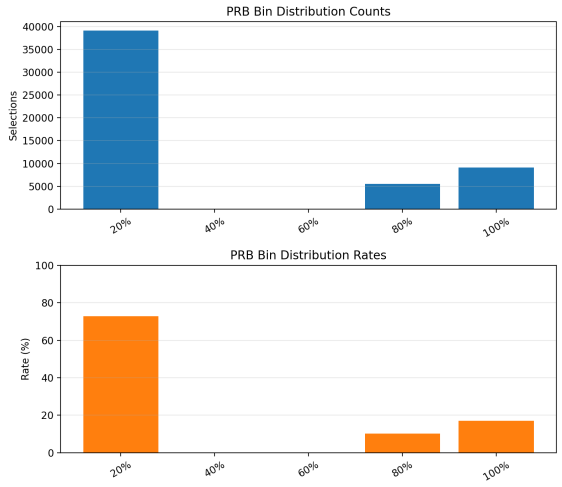


(b) Representative seed-9001 reward timeline for NoEmbedding.

Figure 18: Seed-level reliability and reward stability for the NoEmbedding ablation.



(a) Destination distribution on seed 9001 for NoEmbedding.

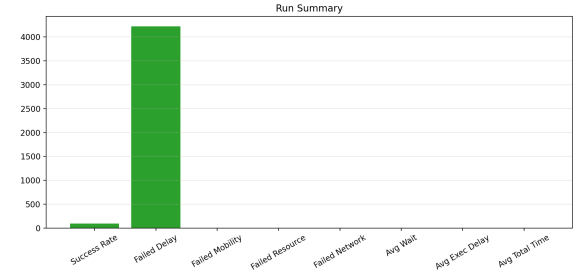


(b) PRB-bin distribution on seed 9001 for NoEmbedding.

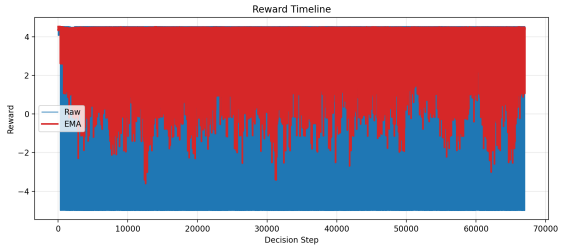
Figure 19: Representative NoEmbedding action behavior under the standard evaluation setting.

FixedPRB: structural failure of the joint scheduler. The FixedPRB ablation changes the outcome qualitatively. Training improves only from 55.90% to 91.52% success across the 40 episodes and plateaus there instead of entering the near-perfect regime. In evaluation, mean success collapses to 92.3822%, mean energy jumps to 309.2645 W, and mean delay failures explode to 5127.67. Unlike NoEmbedding, this is not a small regularization effect; it is a direct loss of scheduling capability. The simulator also becomes much faster to run at 145.41 s per episode on average, but the speedup reflects a degenerate policy with less adaptive resource usage rather than a better optimization process.

The representative seed-9001 charts in Fig. 20 and Fig. 21 reveal the mechanism. The run-summary chart is dominated by the failed-delay bar, and the reward timeline becomes highly oscillatory with dense negative excursions, showing that bad decisions are frequent throughout the episode rather than confined to a few isolated events. The destination distribution also collapses to a two-way split: 43.37% local and 42.29% cloud, with the four edge nodes together receiving only 14.35% of selections. Most decisively, the PRB distribution degenerates to a single point mass, with 100% of offloaded tasks using the 80% bin by construction. This removes the policy's ability to match radio reservation to task urgency and destination quality. The result is a system that underuses edge resources, overconsumes mist energy, and accumulates thousands of deadline misses even though fallback, mobility, and network failures remain at zero.



(a) Representative seed-9001 run summary for FixedPRB.

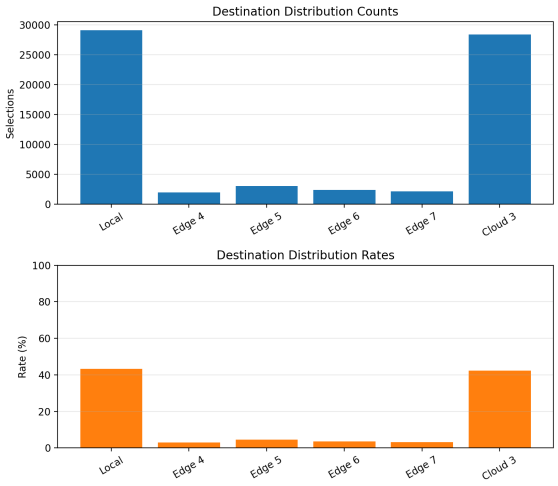


(b) Representative seed-9001 reward timeline for FixedPRB.

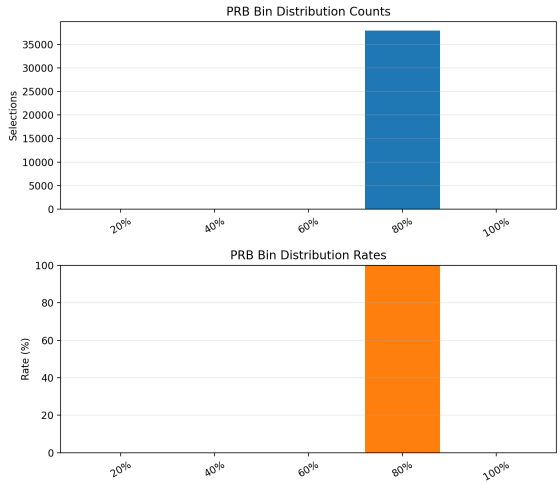
Figure 20: Seed-level reliability and reward instability for the FixedPRB ablation.

The additional diagnostics in Fig. 22 sharpen the contrast between the two ablations. NoEmbedding still tracks destination availability in roughly the same order as the full MAPPO policy and retains an energy profile very close to the main model, which is consistent with its relatively small performance loss. FixedPRB behaves differently: the selection histogram shifts further toward local execution while cloud choice remains high, and the total energy rises almost entirely because mist-device energy grows sharply. This confirms that the fixed-radio ablation does not merely reduce flexibility in the network layer; it changes the global compute-radio balance of the whole system.

The runtime-level resource dynamics in Fig. 23 make the contrast between adap-



(a) Destination distribution on seed 9001 for FixedPRB.



(b) PRB-bin distribution on seed 9001 for FixedPRB.

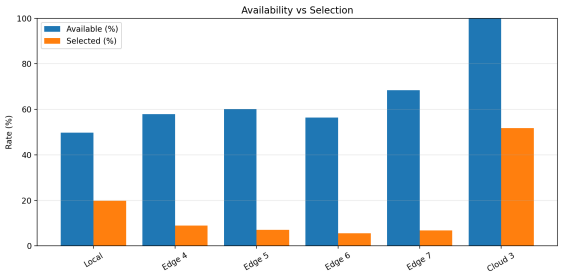
Figure 21: Representative FixedPRB action collapse under the standard evaluation setting.

tive and fixed PRB allocation visually immediate. The full MAPPO policy maintains a moderate, fluctuating PRB reservation that adapts to instantaneous contention, while the FixedPRB variant shows a rigid, uniformly high allocation pattern that saturates the wireless pool during congestion peaks. The task failure timeline in Fig. 24 reinforces this: MAPPO's failure curve stays flat near zero, whereas FixedPRB accumulates failures steadily throughout the episode, with visible acceleration during high-traffic windows. These runtime traces confirm that the aggregate metrics in Tab. 14 are not artifacts of a few bad episodes but reflect a systematic inability of the fixed-radio policy to manage wireless contention.

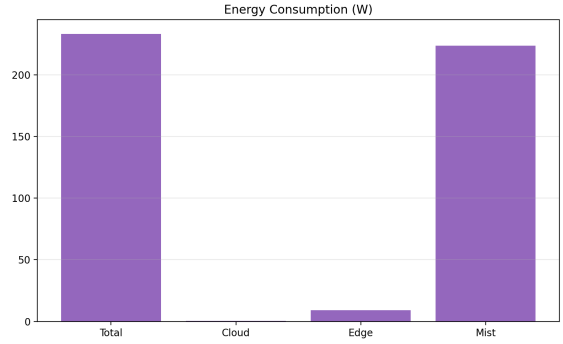
Taken together, the ablations clarify the contribution of each MAPPO component. Adaptive PRB control is the dominant contributor to system quality: removing it destroys both reliability and energy efficiency. Agent embeddings provide a smaller but consistent benefit: removing them does not break the scheduler, but increases residual delay failures and pushes the policy toward more aggressive PRB usage. The MAPPO design therefore gains most of its practical value from joint offloading-and-PRB learning, while explicit agent conditioning refines that shared control and delivers the last fraction of reliability.

5.4 Comparison between Reinforcement Learning and Baseline Algorithms

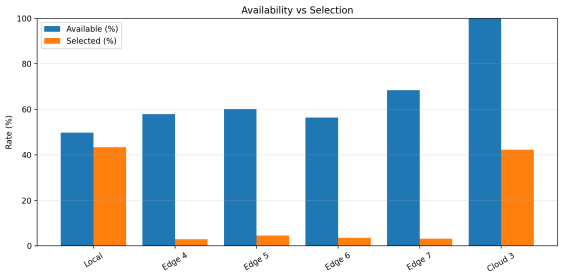
The offline all-algorithm comparison confirms that reinforcement learning decisively outperforms the heuristic baselines under the standard 160-device setting. MAPPO



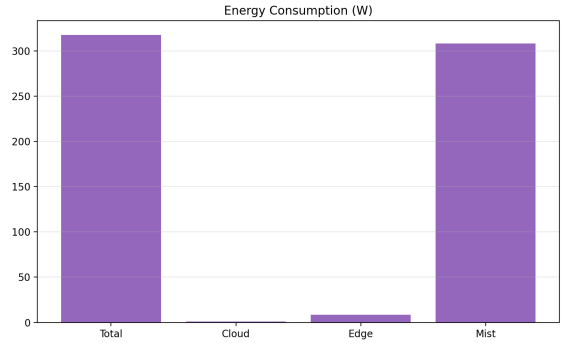
(a) Availability vs. selection for NoEmbedding.



(b) Energy breakdown for NoEmbedding.

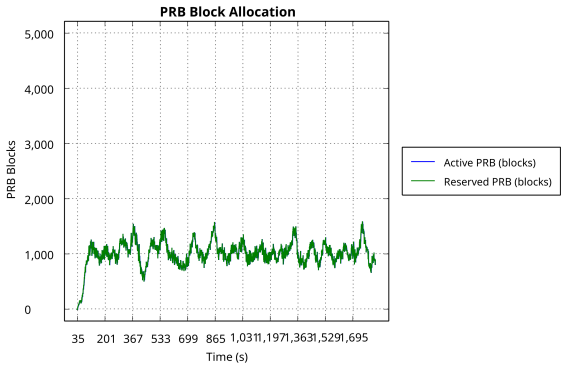


(c) Availability vs. selection for FixedPRB.

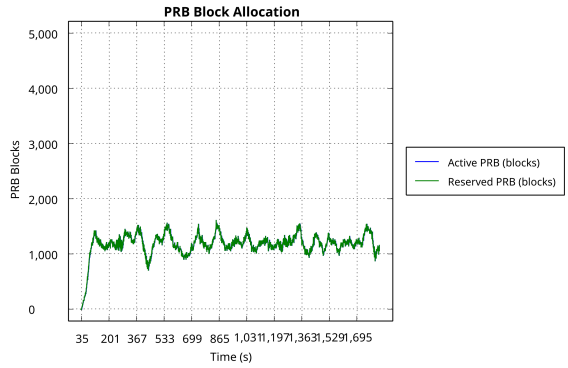


(d) Energy breakdown for FixedPRB.

Figure 22: Additional seed-level diagnostics for the two MAPPO ablations under the standard evaluation setting.

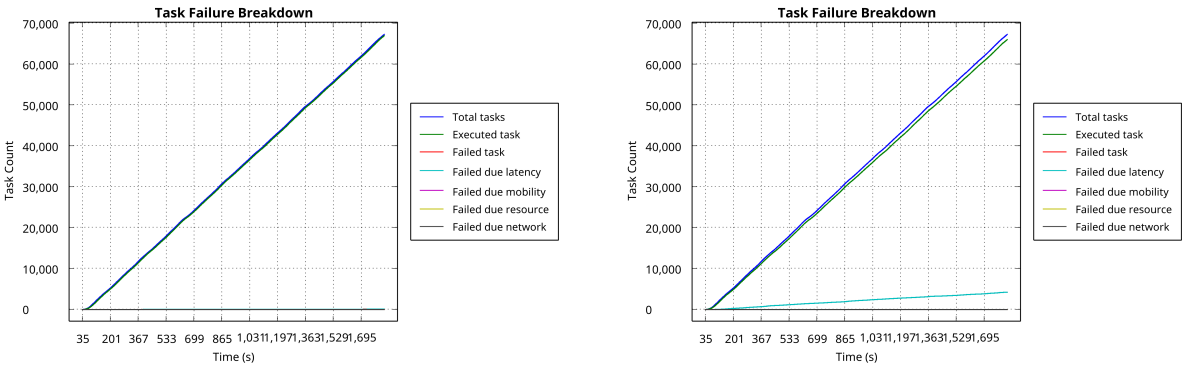


(a) PRB block allocation over time for full MAPPO.



(b) PRB block allocation over time for FixedPRB.

Figure 23: Comparison of PRB block allocation dynamics between the full MAPPO policy (adaptive) and the FixedPRB ablation (fixed at 80%) on seed 9001.



(a) Task failure timeline for full MAPPO.

(b) Task failure timeline for FixedPRB.

Figure 24: Comparison of task failure accumulation between the full MAPPO policy and the FixedPRB ablation on seed 9001.

achieves the highest task success rate at 99.96%, followed closely by PPO at 99.94%. The strongest heuristic remains ROUND_ROBIN at 90.39%, which is still far below both learned policies. The comparison also shows that the RL gain is not obtained by simply consuming more energy than every baseline: MAPPO and PPO both use about 231.3 W, which is substantially lower than ROUND_ROBIN's 298.26 W.

Tab. 15 lists the main metrics for all 10 algorithms, while Fig. 25 visualizes the same comparison. Several patterns are worth emphasizing. First, MAPPO improves success rate by 9.58 percentage points over ROUND_ROBIN while also reducing average total time by 1.2750 s and energy consumption by 67.00 W. Second, compared with PPO, MAPPO is slightly more reliable and slightly lower in energy, whereas PPO is slightly faster in average total time by 0.0670 s. Third, CLOUD appears attractive in latency and energy, yet its success rate remains only 64.64%, so it is not a viable scheduling policy when completion reliability matters. Fourth, LOCAL fails mainly because 33,630 tasks cannot be executed under local-only capacity constraints, which highlights the need for coordinated multi-tier offloading.

The aggregate bars are reinforced by the simulator-side runtime charts in Fig. 26 and Fig. 27. MAPPO and PPO both remain near 100% success throughout the full 30-minute horizon, with only very small short-window oscillations. ROUND_ROBIN behaves differently: its total success gradually settles near 90% and its 30-second window fluctuates much more strongly, which is consistent with the larger number of delay failures in Tab. 15. The destination-distribution plots explain why. MAPPO and PPO both learn a cloud-dominant but still genuinely three-tier policy, keeping roughly one-fifth of the workload local and distributing the remaining non-cloud traffic across multiple edge sites. ROUND_ROBIN, by contrast, maintains a much larger local fraction and a flatter, more mechanical edge split, which limits its ability to relieve deadline pressure when contention rises.

Algorithm	Success (%)	Avg. Total Time (s)	Energy (W)	Delay Failures	Avg. CPU (%)	Avg. BW (Mbps)
RANDOM	41.05	4.5641	149.39	39 630	6.56	50.79
LOCAL	45.36	7.3408	256.77	3 133	41.03	–
EDGE	42.66	5.5131	135.97	38 552	3.42	68.91
CLOUD	64.64	4.3210	135.68	23 794	0.85	34.26
ROUND_ROBIN	90.39	5.8304	298.26	6 452	43.97	77.26
TRADE_OFF	60.45	5.0701	203.79	26 585	31.94	57.06
LAT_EN_AWARE	49.80	4.6286	190.54	33 752	29.00	60.05
TEST	79.91	6.4780	196.90	13 435	30.93	44.68
PPO	99.94	4.4884	231.38	36	21.66	4.07
MAPPO	99.96	4.5554	231.26	20	21.66	4.07

Table 15: Offline comparison of all algorithms under the standard 160-device scenario. Avg. CPU is the overall VM utilization; Avg. BW is the mean bandwidth per task. LOCAL uses no network offloading, so bandwidth is not applicable. A horizontal rule separates the RL policies from the heuristic baselines.

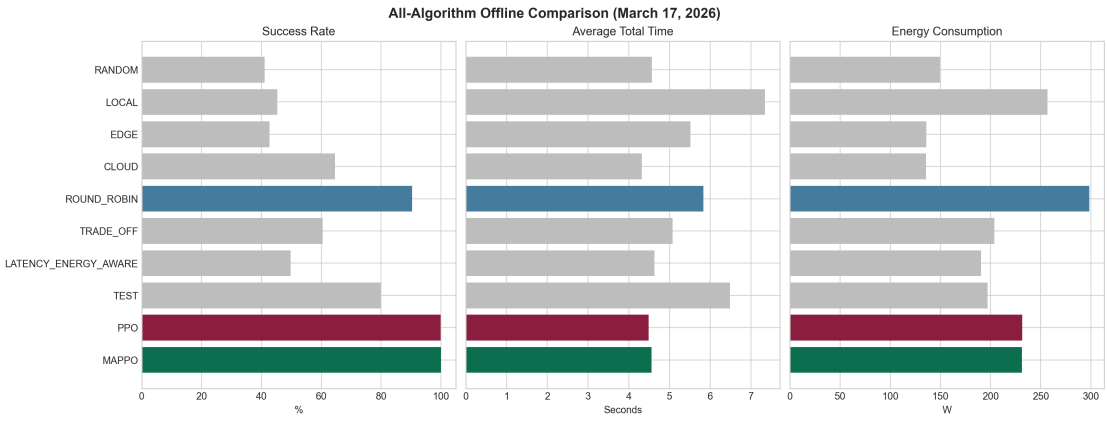
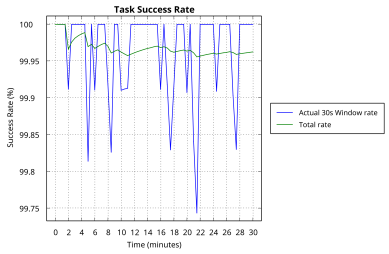
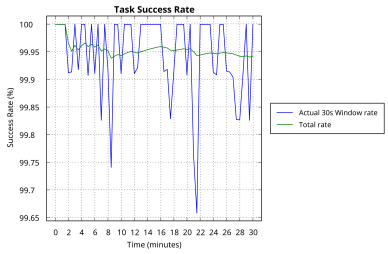


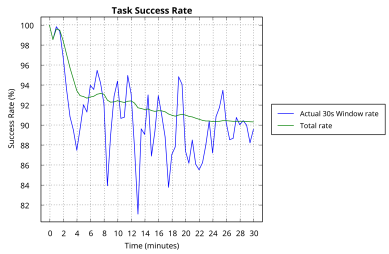
Figure 25: Comparison of all algorithms in terms of success rate, average total time, and energy consumption.



(a) MAPPO.

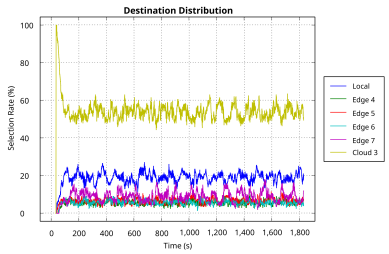


(b) PPO.

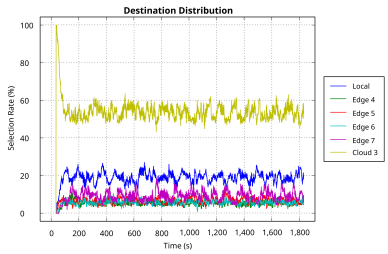


(c) ROUND_ROBIN.

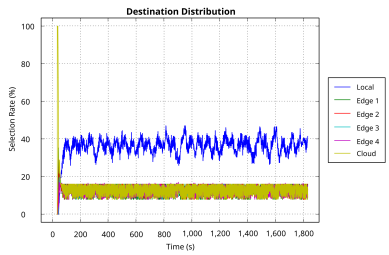
Figure 26: Representative task-success trajectories over the 30-minute offline comparison run for MAPPO, PPO, and ROUND_ROBIN.



(a) MAPPO.



(b) PPO.



(c) ROUND_ROBIN.

Figure 27: Representative destination-distribution trajectories for MAPPO, PPO, and ROUND_ROBIN in the offline comparison run.

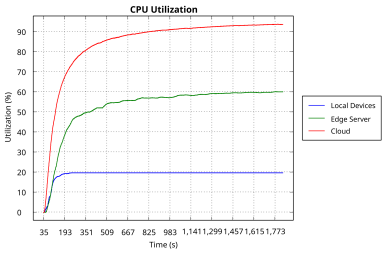
From a systems perspective, the main benefit of reinforcement learning is coordinated tier usage. The heuristic policies either overload one tier, ignore PRB pressure, or trade off latency and energy only through fixed rules. By contrast, PPO and MAPPO both learn to mix local, edge, and cloud usage while adapting to queueing pressure and wireless contention. Tab. 16 quantifies this tier-level breakdown. MAPPO distributes 35,072 tasks to cloud, 18,583 to edge, and 13,628 locally, achieving a balanced three-tier pattern. PPO follows a nearly identical distribution. In contrast, ROUND_ROBIN keeps 25,132 tasks local and sends only 8,433 to cloud, which explains its higher energy and lower success rate. The single-tier algorithms (LOCAL, EDGE, CLOUD) each concentrate all traffic on one layer, leading to severe overload or underutilization of the remaining tiers.

Algorithm	Cloud Tasks	Edge Tasks	Mist Tasks	Cloud Success	Edge Success
RANDOM	20 451	46 193	639	17 850	9 134
LOCAL	0	0	33 653	–	–
EDGE	0	67 283	0	–	28 705
CLOUD	67 283	0	0	43 489	–
ROUND_ROBIN	8 433	33 718	25 132	8 433	28 725
TRADE_OFF	17 040	40 854	9 389	17 038	14 245
LAT_EN_AWARE	15 584	43 891	7 808	15 584	10 112
TEST	23 355	36 018	7 910	20 365	25 489
PPO	34 928	18 697	13 658	34 928	18 676
MAPPO	35 072	18 583	13 628	35 072	18 576

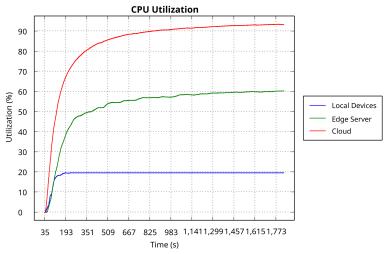
Table 16: Per-tier task distribution and success counts. RL policies achieve near-perfect success on every tier, while heuristics suffer heavy losses on the tiers they overload.

CPU utilization and delay patterns. The CPU utilization and delay charts in Fig. 28 and Fig. 29 reveal the operational mechanisms behind the aggregate metrics. MAPPO and PPO maintain moderate, stable CPU utilization across all tiers, with cloud servers loaded at around 93% and edge servers at around 60%, while mist devices stay below 20%. This balanced loading pattern avoids both idle waste and congestion-induced queuing. ROUND_ROBIN, by contrast, pushes overall CPU utilization to 43.97% with edge servers near saturation, which creates queuing delays that cascade into deadline misses. The TRADE_OFF heuristic shows a similar but less severe pattern, with cloud servers at 65% and edge at 95%.

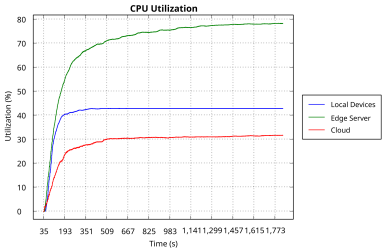
The delay distributions tell a complementary story. MAPPO and PPO concentrate task completion times in a narrow band well below the deadline threshold, with very few tasks in the tail. ROUND_ROBIN spreads delays more broadly, and the rightward tail extends into the deadline-violation region, which directly accounts for its 6,452 delay failures. The TRADE_OFF heuristic shows an even wider delay spread, consistent with its lower success rate.



(a) MAPPO.

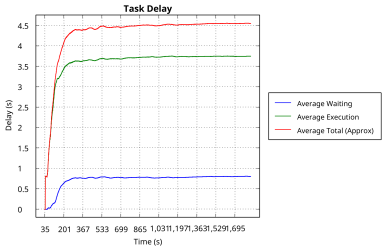


(b) PPO.

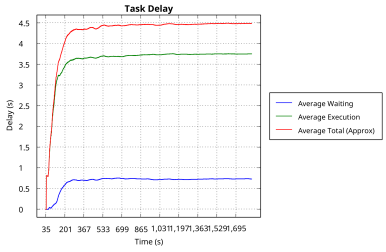


(c) ROUND_ROBIN.

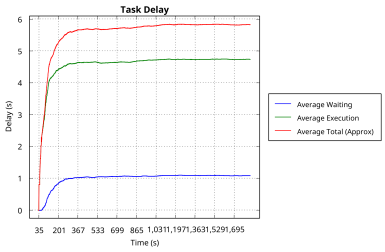
Figure 28: CPU utilization over time for MAPPO, PPO, and ROUND_ROBIN in the offline comparison run.



(a) MAPPO.



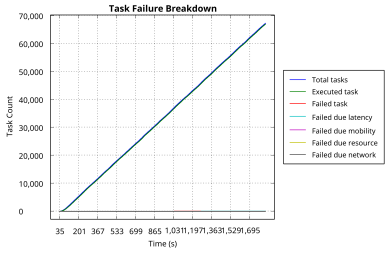
(b) PPO.



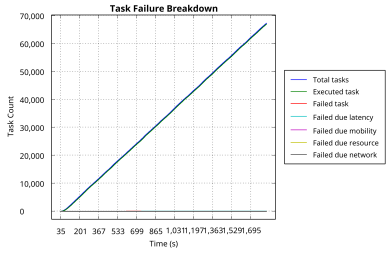
(c) ROUND_ROBIN.

Figure 29: Delay distributions for MAPPO, PPO, and ROUND_ROBIN in the offline comparison run.

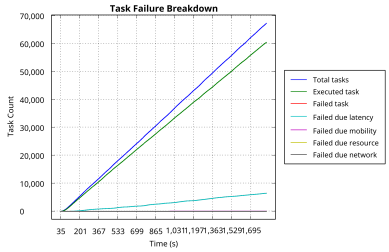
Task failure accumulation. The task failure timelines in Fig. 30 provide the most direct visual evidence of the RL advantage. Both MAPPO and PPO maintain essentially flat failure curves throughout the 30-minute horizon, with only 20 and 36 delay failures respectively. ROUND_ROBIN accumulates failures at a roughly linear rate, reaching 6,452 by the end of the simulation. This linear accumulation pattern indicates that ROUND_ROBIN's failures are not caused by transient congestion spikes but by a persistent mismatch between its static scheduling rule and the dynamic resource landscape.



(a) MAPPO.



(b) PPO.



(c) ROUND_ROBIN.

Figure 30: Task failure accumulation over time for MAPPO, PPO, and ROUND_ROBIN in the offline comparison run.

Reinforcement learning is therefore not merely competitive with heuristics; it is clearly superior across every dimension examined: success rate, latency, energy efficiency, CPU utilization balance, and temporal stability of the failure rate.

5.5 Analysis of PPO

PPO serves as the closest learning-based baseline to MAPPO because it keeps the same environment and dual action heads while removing the explicit multi-agent parameterization. In the updated implementation, the controlled difference is now clearer than before: MAPPO uses both an agent embedding and a telemetry-augmented critic state, whereas PPO still uses a single shared actor and the original 27-dimensional critic state. For this reason, all direct MAPPO–PPO comparisons are collected in this subsection rather than being mixed into the standard MAPPO subsection.

Training comparison. Under the standard training setup, PPO behaves very similarly to MAPPO during learning. PPO starts from 57.94% success in episode 1, rises to 99.93% by episode 40, crosses 95% success by episode 6, and reaches 99% by episode 12. Its average simulator wall time is 223.48 s per episode, making it about 4.26 s faster than MAPPO per episode on average. Fig. 31 and Tab. 17 summarize the cross-model training dynamics.

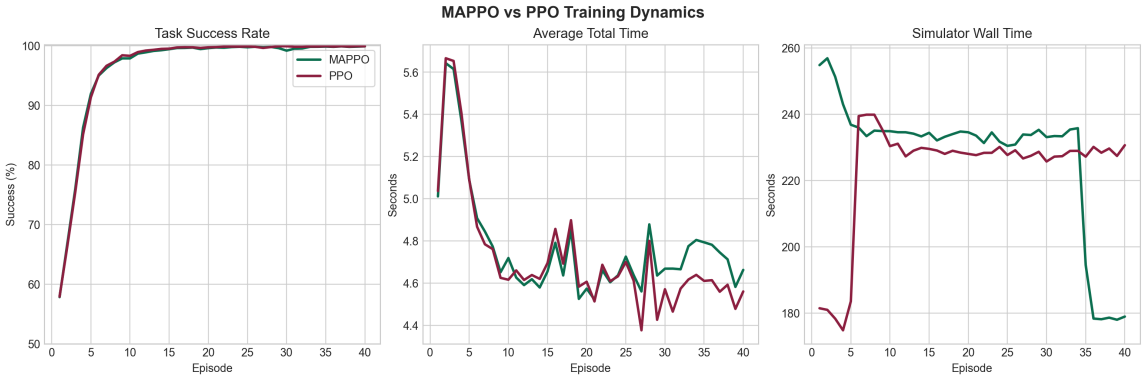


Figure 31: Training comparison between MAPPO and PPO under the standard 40-episode setting.

Episode	MAPPO		PPO	
	Success (%)	Avg. Total Time (s)	Success (%)	Avg. Total Time (s)
1	57.85	5.0108	57.94	5.0363
5	91.96	5.0888	91.38	5.0898
10	97.90	4.7186	98.32	4.6157
20	99.62	4.5733	99.75	4.6061
30	99.19	4.6681	99.93	4.5705
40	99.91	4.6622	99.93	4.5599

Table 17: Selected training checkpoints for MAPPO and PPO under the standard setting.

Evaluation comparison. The major difference between MAPPO and PPO is now much smaller than initially expected. PPO remains extremely stable across all three seeds, with success rates between 99.92% and 99.96%. The three-seed mean success rate is 99.9436%, the standard deviation is 0.0137, the mean total time is 4.4665 s, and the mean energy consumption is 227.5745 W. MAPPO is only slightly behind on the three-seed mean, with 99.9237% success and 0.0238 standard deviation, while winning the single-seed offline all-algorithm comparison by a narrow margin. The aggregated comparison is summarized in Tab. 18 and Fig. 32.

The figure makes the trade-off clear. PPO still retains a small edge in three-seed mean success rate, mean total time, and variance, but the gap is now extremely small.

Model	Mean Success (%)	Std. Success	Mean Total Time (s)	Mean Energy (W)	Train Wall Time (s/ep
MAPPO	99.9237	0.0238	4.5284	227.6360	227.74
PPO	99.9436	0.0137	4.4665	227.5745	223.48

Table 18: Summary comparison of MAPPO and PPO under the finalized three-seed evaluation.

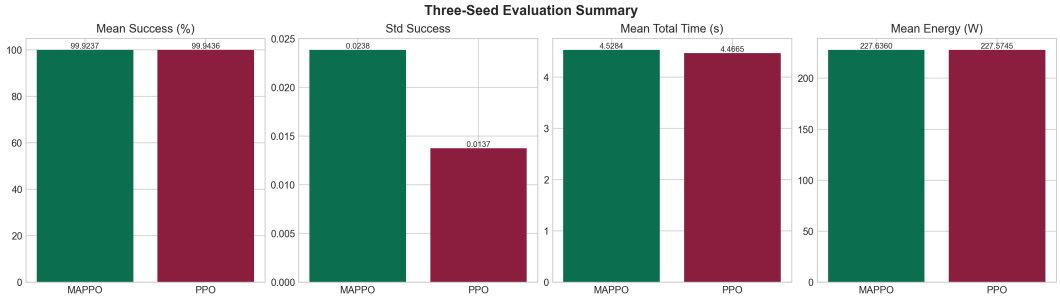


Figure 32: Final evaluation summary of MAPPO and PPO over three test seeds.

At the same time, MAPPO is no longer a high-variance outlier: it reaches near-perfect success on every seed, removes fallback events entirely, and slightly surpasses PPO in the offline fixed-seed comparison.

One limitation should be noted: the PPO run does not preserve per-decision trajectory CSV files, so a detailed action-level analysis of PPO destination selection and PRB usage cannot be reproduced at the same granularity as MAPPO. This does not affect the aggregate evaluation metrics but prevents a fully symmetric behavioral comparison.

Taken together, the PPO results clarify the role of multi-agent modeling. On-policy reinforcement learning itself is highly effective for this scheduling problem. At the same time, MAPPO is competitive with PPO in every aggregate metric while winning the offline fixed-seed comparison. The ablation results make the architectural story clearer: adaptive PRB control is indispensable, and explicit agent conditioning provides a smaller but measurable reliability gain. PPO is the slightly safer choice if only cross-seed mean performance is considered, but MAPPO is a technically credible and practically strong multi-agent scheduler.

6 Conclusion

6.1 Summary

This report studied joint task offloading and PRB allocation for a three-tier 6G edge-cloud computing architecture. The problem was formulated as a Dec-POMDP and implemented through a customized PureEdgeSim environment that supports PRB-aware networking, device-level observations, Java–Python co-simulation, and on-policy reinforcement learning. Both MAPPO and PPO significantly outperform the heuristic baselines. In the offline single-seed all-algorithm comparison, MAPPO reaches a task success rate of 99.96%, PPO reaches 99.94%, and the strongest heuristic, ROUND_ROBIN, remains at 90.39%.

Under the standard 40-episode training setting, MAPPO converges quickly, exceeds 99% task success by episode 13, and evaluates stably on all three test seeds with a mean success rate of 99.9237%, zero fallback events, a mean total time of 4.5284 s, and a mean energy consumption of 227.6360 W. PPO retains a small edge in cross-seed mean performance, with 99.9436% success, a standard deviation of 0.0137, and a mean total time of 4.4665 s, but the gap is very small. The ablation study clarifies the architectural contribution: removing agent embeddings only reduces mean success to 99.8148%, whereas fixing PRB requests collapses performance to 92.3822% and raises mean energy to 309.2645 W. The device-count scalability study further shows that MAPPO scales gracefully from 80 to 240 devices (maintaining success rates above 99.4%), with the 320-device collapse attributable to infrastructure saturation rather than algorithmic failure. The main conclusion is therefore threefold: on-policy reinforcement learning is clearly effective for this problem, adaptive PRB control is essential to the final scheduler quality, and explicit multi-agent conditioning provides a smaller but still measurable reliability gain on top of that joint control.

6.2 Limitations

Several limitations should be noted. First, the study is still simulation-based, so there remains a gap between the modeled wireless environment and a real 5G or 6G deployment. Second, although the device-count scalability study covers four configurations and the ablation study isolates two controlled variants, a broader architectural search (e.g., varying critic input design, reward coefficients, or PRB bin granularity) would strengthen the conclusions. Third, the reward function is hand-designed and may not be optimal for every workload mix or operator objective. Fourth, while the training pipeline already supports GAE, the event-driven task setting still simplifies some long-horizon temporal dependencies that would become more pronounced in persistent real-world traffic traces.

Another practical limitation is artifact completeness. MAPPO preserves detailed trajectory CSV files and supports fine-grained action analysis, but the finalized PPO run

does not preserve the same trajectory exports. This does not invalidate the aggregate PPO metrics, but it does prevent a fully symmetric behavioral audit. Finally, although the preserved three seeds no longer show MAPPO collapse, broader robustness validation across more seeds, traffic mixes, and topology perturbations is still necessary before claiming general superiority.

6.3 Future Work

There are several natural directions for future work.

- Expand the ablation family beyond NoEmbedding and FixedPRB to cover additional architectural variants such as critic input design, reward coefficient sensitivity, and PRB bin granularity.
- Investigate robustness-oriented improvements for MAPPO, including broader seed sweeps, checkpoint selection based on validation stability, and alternative reward shaping for safer PRB allocation behavior.
- Regenerate PPO trajectory exports so that destination-choice and PRB-behavior analysis can be compared directly against MAPPO at the action level.
- Scale the simulator to denser deployments and larger edge clusters in order to test how coordinated policies behave under heavier multi-agent coupling, particularly beyond the 240-device infrastructure saturation boundary identified in the current study.
- Integrate the scheduling framework with richer traffic traces or a hardware testbed to reduce the simulation-to-reality gap.

Overall, the results indicate that on-policy learning is a practical and promising approach for coordinated offloading and radio-resource control in hierarchical edge-cloud systems. The current evidence strongly supports reinforcement learning over rule-based scheduling, identifies adaptive PRB control as the key design factor, and shows that MAPPO is now a technically credible multi-agent scheduler rather than an unstable curiosity, even though PPO remains a very strong benchmark and broader validation is still required.

References

[1] W. Saad, M. Bennis, and M. Chen, "A vision of 6g wireless systems: Applications, trends, technologies, and open research problems," **IEEE Network**, vol. 34, no. 3, pp. 134–142, 2020. pages 1, 2

[2] M. Satyanarayanan, "The emergence of edge computing," **Computer**, vol. 50, no. 1, pp. 30–39, 2017. pages 1, 2

[3] Y. Mao, C. Y. Zhang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," **IEEE Communications Surveys & Tutorials**, vol. 19, no. 4, pp. 2322–2358, 2017. pages 1, 2

[4] Z. Luo and X. Dai, "Reinforcement learning-based computation offloading in edge computing: Principles, methods, challenges," **Alexandria Engineering Journal**, vol. 108, pp. 89–107, 2024. pages 3, 4

[5] A. Robles-Enciso and A. F. Skarmeta, "A multi-layer guided reinforcement learning-based tasks offloading in edge computing," **Computer Networks**, vol. 220, p. 109476, 2023. pages 3, 4, 5, 6, 7, 8, 11, 12

[6] H. Lu, C. Gu, F. Luo, W. Ding, S. Zheng, and Y. Shen, "Optimization of task offloading strategy for mobile edge computing based on multi-agent deep reinforcement learning," **IEEE Access**, vol. 8, pp. 202 573–202 584, 2020. pages 3, 4, 6

[7] H. Ke, H. Wang, and H. Sun, "Multi-agent deep reinforcement learning-based partial task offloading and resource allocation in edge computing environment," **Electronics**, vol. 11, no. 15, p. 2394, 2022. pages 3, 4, 6, 20

[8] Y. Sun and Q. He, "Computational offloading for mec networks with energy harvesting: A hierarchical multi-agent reinforcement learning approach," **Electronics**, vol. 12, no. 6, p. 1304, 2023. pages 3, 4, 5, 6, 20

[9] Z. Aghapour, S. Sharifian, and H. Taheri, "Task offloading and resource allocation algorithm based on deep reinforcement learning for distributed AI execution tasks in IoT edge computing environments," **Computer Networks**, vol. 223, p. 109577, 2023. pages 4, 6

[10] B. Cao, Y. Yi, Z. Zeng, H. Ye, and B. Tang, "A deep Q-network-based edge service offloading in cloud–edge–terminal environment," **The Journal of Supercomputing**, vol. 81, p. 758, 2025. pages 4

[11] C. Mechalikh, H. Taktak, F. Moussa, and S. Pllana, "Pureedgesim: A simulation framework for performance evaluation of cloud, edge and mist computing environments," **Computer Science and Information Systems**, 2021. pages 5, 7, 9

[12] C. Mechalikh, A. E. H. Silem, Z. Safavifar, and F. Golpayegani, "Quality matters: A comprehensive comparative study of edge computing simulators," **Simulation Modelling Practice and Theory**, vol. 138, p. 103042, 2025. pages 5, 7

[13] I. Abid and T. N. Gia, "Eisim: Simulating ai-driven management of large-scale wireless edge computing systems," 2023. pages 5

[14] alb1183, "Ml-rl-pureedgesim," https://github.com/alb1183/ML-RL-PureEdgeSim, 2026, gitHub repository, accessed March 16, 2026. pages 5

[15] C. Yu, A. Velu, E. Vinitsky, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," in **Advances in Neural Information Processing Systems**, vol. 35, 2022. pages 5, 25

[16] O. Giwa, J. du Toit, J. Shock, and T. Awodumila, "Optimisation of resource allocation in heterogeneous wireless networks using deep reinforcement learning," **arXiv preprint arXiv:2509.25284**, 2025. pages 5, 6

[17] A. Robles-Enciso and A. F. Skarmeta, "Task offloading in computing continuum using collaborative reinforcement learning," in **Global Internet of Things Summit**, ser. Lecture Notes in Computer Science, vol. 13533, 2022, pp. 82–95. pages 5, 7, 8, 11, 12

[18] C. Mechalikh, "Pureedgesim," https://github.com/CharafeddineMechalikh/PureEdgeSim, 2026, gitHub repository, accessed March 16, 2026. pages 7

[19] M. C. d. Silva Filho, R. Oliveira, C. Monteiro da Silva, G. L. Oliveira, E. Oikoski, C. Costa, Y. C. Rocha, and M. D. de Assuncao, "Cloudsim plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness," in **IFIP/IEEE Symposium on Integrated Network and Service Management**, 2017, pp. 400–406. pages 7

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. pages 25

A Supplementary Materials

A.1 TCP JSON Message Protocol

The Java-Python bridge uses newline-delimited JSON over a localhost TCP socket. The message types exchanged by `RLEnvServer` and the Python client are summarized in Tab. 19.

Message type	Direction		Main fields
`marl_config`	Java Python	→	`num_agents`, `num_destinations`, `agent_obs_dim`, `dest_feat_dim`, `state_dim`, `prb_bins`, destination labels, PRB-bin labels
`marl_turn_obs`	Java Python	→	`step_id`, `agent_id`, `agent_obs`, `dest_features`, `dest_mask`, `state`
`marl_action`	Python Java	→	`step_id`, `dest_action`, `prb_action`
`marl_transition`	Java Python	→	`step_id`, `reward`, `done`, requested and executed destination actions, requested and executed PRB actions, `dest_fallback`
`marl_episode_end`	Java Python	→	`done`, `episode_index`, `final_state`, `fallback_count`
`control`	Python Java	→	Control commands such as `terminate`

Table 19: TCP JSON protocol used by the co-simulation interface.

Because the protocol is message-oriented and synchronous, it is straightforward to debug by logging raw JSON lines. This was particularly useful during development of fallback handling and offline inference mode.

A.2 Hyperparameter Notes

The MAPPO and PPO training scripts share the same optimizer-level hyperparameters, including learning rates, clipping factor, minibatch size, and entropy annealing schedule. The main structural difference is that MAPPO uses an agent embedding and a centralized critic over the telemetry-augmented 41-dimensional state, whereas PPO removes the embedding and relies on a single shared actor representation with the original 27-dimensional critic state. Both actors still use the same dual-head action design for destination and PRB selection.

For reproducibility, the final quantitative results reported in Section 5 are based on 40-episode training artifacts for both MAPPO and PPO, each followed by three evaluation seeds: 9001, 9002, and 9003. The PPO training run uses the same episode count and hyperparameters as MAPPO.

A.3 Reproducibility Notes

The report is based on the following artifact layout.

- The thesis template is stored in the report workspace.
- The simulator and learning code are stored in the companion project workspace.
- The 40-episode MAPPO and PPO training runs are preserved under the `Results/` directory, with structured subdirectories for each experiment.
- The three-seed evaluation summaries for MAPPO, PPO, and all ablation variants are stored as `run_summary.json` files under the corresponding evaluation subdirectories.
- The device-count scalability study (80, 160, 240, 320 devices) is preserved under `Results/ablation_devices/`, and the component ablations (NoEmbedding, FixedPRB) are preserved under `Results/ablation_NoEmbedding/` and `Results/ablation_FixedP`
- Heuristic baseline results from the offline all-algorithm comparison are preserved as both simulator CSV exports and dashboard images.

In practical use, the workflow is: prepare the PureEdgeSim configuration, launch training mode for MAPPO or PPO through the Python scripts, save model checkpoints, then switch to offline inference mode or batch evaluation mode to collect summary metrics and dashboard plots.